开发者社区

DTK

DTK 26.10
hipSOLVER 库使用手册

DTK-261000-029-0915-944

目　录

1 hipSOLVER 库简介 1
2 环境要求 2
3 核心概念 2
3.1 hipSOLVER 使用介绍 2
3.2 简单 demo 2
4 API 详解 8
4.1 hipSolverDn 8
4.1.1 hipsolverDnCreate 8
4.1.2 hipsolverDnDestroy 8
4.1.3 hipsolverDnSetStream 8
4.1.4 hipsolverDnGetStream 8
4.1.5 hipsolverDn<t>orgbr() 9
4.1.6 hipsolverDn<t>orgqr() 12
4.1.7 hipsolverDn<t>orgtr() 16
4.1.8 hipsolverDn<t>ormqr() 19
4.1.9 hipsolverDn<t>ormtr() 24
4.1.10 hipsolverDn<t>gebrd() 28
4.1.11 hipsolverDn<t><t>gels() 31
4.1.12 hipsolverDn<t><t>gesv() 41
4.1.13 hipsolverDn<t>geqrf() 51
4.1.14 hipsolverDn<t>getrf() 54
4.1.15 hipsolverDn<t>getrs() 56
4.1.16 hipsolverDn<t>potrf() 58
4.1.17 hipsolverDn<t>potrfBatched() 61
4.1.18 hipsolverDn<t>potrs() 62
4.1.19 hipsolverDn<t>potrsBatched() 64
4.1.20 hipsolverDn<t>potri() 66
4.1.21 hipsolverDn<t>gesvd() 69

开发者社区

4.1.22 hipsolverDn<t>gesvdj() 73
4.1.23 hipsolverDn<t>gesvdjBatched() 78
4.1.24 hipsolverDn<t>gesvdaStridedBatched() 83
4.1.25 hipsolverDn<t>syevd() 89
4.1.26 hipsolverDn<t>syevdx() 93
4.1.27 hipsolverDn<t>syevj() 98
4.1.28 hipsolverDn<t>syevjBatched() 102
4.1.29 hipsolverDnXsyevBatched() 106
4.1.30 hipsolverDn<t>sygvd() 108
4.1.31 hipsolverDn<t>sygvdx() 112
4.1.32 hipsolverDn<t>sygvj() 118
4.1.33 hipsolverDn<t>sytrd() 122
4.1.34 hipsolverDn<t>sytrf() 126
4.1.35 hipsolverDnIRSXgesv () 128
4.1.36 hipsolverDnIRSXgels () 130
4.1.37 hipsolverDnXgesvdp() 131
4.1.38 hipsolverDnXgesvdr() 133
4.1.39 hipsolverDnXtrtri() 135
4.1.40 hipsolverDnXsytrs() 136
4.1.41 hipsolverDnXgeev() 138
4.1.42 hipsolverDnXlarft() 140
4.2 hipSolverSp 141
4.2.1 hipsolverSpCreate() 141
4.2.2 hipsolverSpDestroy() 141
4.2.3 hipsolverSpSetStream() 142
4.2.4 hipsolverSpGetStream() 142
4.2.5 hipsolverSp <t>csrlsvlu() 142
4.2.6 hipsolverSp <t>csrlsvqr() 144
4.2.7 hipsolverSp <t>csrlsvchol() 146
4.2.8 hipsolverSp <t>csrlsqvqr() 148

开发者社区

4.2.9 hipsolverSp <t>csreigvsi() 151
4.2.10 hipsolverSp <t>csreigs() 153
4.2.11 hipsolverSpXcsrsymrcm() 155
4.2.12 hipsolverSpXcsrsymmdq() 156
4.2.13 hipsolverSpXcsrsymamd() 157
4.2.14 hipsolverSpXcsrmetisnd() 157
4.2.15 hipsolverSpXcsrzfd() 158
4.2.16 hipsolverSpXcsrperm() 160
4.2.17 hipsolverSpXcsrqrBatched() 161
4.2.18 hipsolverspXcsrissymHost () 166
4.3 hipSolverRf 166
4.3.1 hipsolverRfAccessBundledFactorsDevice() 166
4.3.2 hipsolverRfAnalyze() 167
4.3.3 hipsolverRfSetupDevice() 167
4.3.4 hipsolverRfSetupHost() 168
4.3.5 hipsolverRfCreate() 169
4.3.6 hipsolverRfDestroy() 169
4.3.7 hipsolverRfExtractBundledFactorsHost() 170
4.3.8 hipsolverRfExtractSplitFactorsHost() 170
4.3.9 hipsolverRfGet_Algs 171
4.3.10 hipsolverRfGetAlgs 171
4.3.11 hipsolverRfGetMatrixFormat 171
4.3.12 hipsolverRfGetNumericBoostReport 172
4.3.13 hipsolverRfGetNumericProperties 172
4.3.14 hipsolverRfGetResetValuesFastMode 172
4.3.15 hipsolverRfRefactor 173
4.3.16 hipsolverRfResetValues 173
4.3.17 hipsolverRfSetAlgs 174
4.3.18 hipsolverRfSetMatrixFormat 174
4.3.19 hipsolverRfSetNumericProperties 175

4.3.20 hipsolverRfSetResetValuesFastMode 175
4.3.21 hipsolverRfSolve 175
4.3.22 hipsolverRfBatchSetupHost 176
4.3.23 hipsolverRfBatchAnalyze 177
4.3.24 hipsolverRfBatchRefactor 178
4.3.25 hipsolverRfBatchSolve 178
4.3.26 hipsolverRfBatchZeroPivot 179
4.4 hipSolverMG 179
4.4.1 hipsolverMgCreate() 179
4.4.2 hipsolverMgDestroy() 179
4.4.3 hipsolverMgDeviceSelect() 180
4.4.4 hipsolverMgCreateDeviceGrid() 180
4.4.5 hipsolverMgDestroyGrid() 181
4.4.6 hipsolverMgCreateMatrixDesc() 181
4.4.7 hipsolverMgDestroyMatrixDesc() 181
4.4.8 hipsolverMgPotrf() 182
4.4.9 hipsolverMgPotrs() 183
4.4.10 hipsolverMgPotri() 184
4.4.11 hipsolverMgGetrf() 185
4.4.12 hipsolverMgGetrs() 186
4.4.13 hipsolverMgSyevd() 188
5 调试与分析 189
5.1 环境变量 189
开发者社区

版本更新：

- DTK-25.04.1：
  - 创建文档。

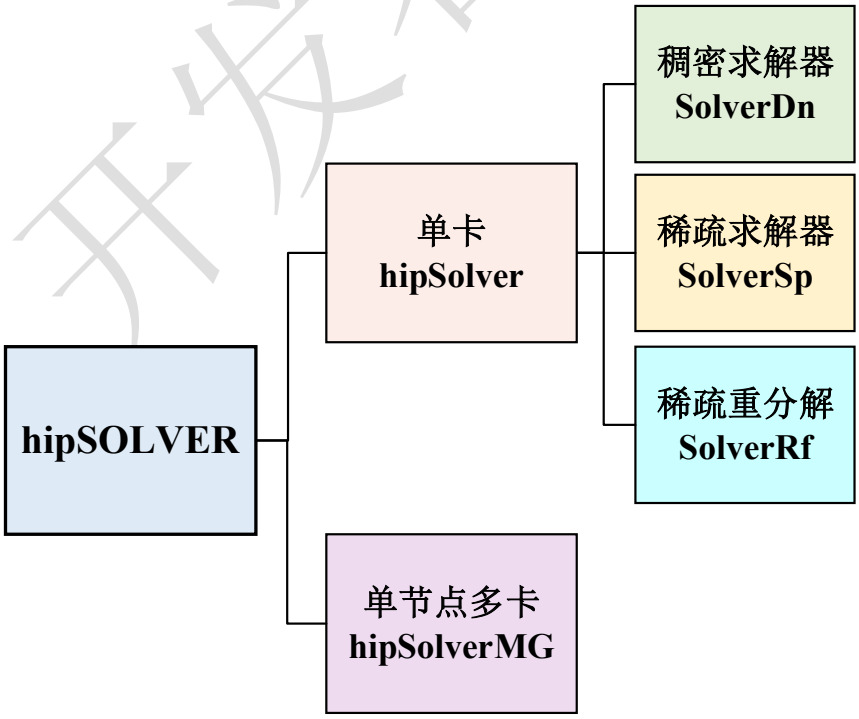
开发者社区

1 hipSOLVER 库简介

hipSOLVER 是 DTK 中用于稠密和稀疏矩阵分解和线性方程组求解的 DCU 加速库。hipSOLVER 基于 hipBLAS 和 hipSPARSE 库，由两部分组成：hipSolver 和 hipSolverMG，分别应用于单卡求解和单节点多卡求解。

hipSolver 提供类似于 LAPACK 的实用功能，例如用于稠密矩阵的常见矩阵分解和三角求解器、稀疏最小二乘求解器和特征值求解器，主要包含 hipSolverDn、hipSolverSp 和 hipSolverRf 三个模块，hipSolver 将三个独立的组件整合到一个框架下：hipSolverDn 用于处理稠密矩阵分解和线性求解（例如 LU、QR、SVD 和 LDLT），同时提供矩阵和向量置换功能；hipSolverSp 提供了一组基于稀疏 QR 分解的稀疏求解 API。并非所有矩阵都具有良好的稀疏模式以实现分解的并行性，因此 hipSolverSp 库还提供了 CPU 端的 API 处理此类问题；hipSolverRf 一个稀疏重构模块，在求解稀疏模式保持不变，仅系数发生变化的批量矩阵算例时，可以提供非常好的性能。

hipSolverMG 是 DCU 加速的 ScaLAPACK。目前，hipSolverMG 支持一维列块循环布局，并提供对称特征值求解器。



开发者社区

2 环境要求

硬件要求：DCU 以及配套驱动，多 DCU 系统；

软件栈要求：DTK（优先使用高版本 DTK）、MPI 库；

运行环境配置：多节点环境要求每个机器的配置完全相同，DCU 可见性，可通过 HIP_VISIBLE_DEVICES 设置。

3 核心概念

3.1 hipSOLVER 使用介绍

hipSolverDn、hipSolverSp、hipSolverMG 均支持 float、double、hipFloatComplex 和 hipDoubleComplex 4 种数据类型，hipSolverRf 只支持 double 类型，其中 float 和 double 是标准 C 数据类型，hipFloatComplex 和 hipDoubleComplex 是从 hip_complex.h 中导出的自定义类型。

hipSOLVER 各个模块都有专用的上下文句柄，hipSolverDn、hipSolverSp、hipSolverRf 和 hipSolverMG 对应的句柄分别为：hipsolverDnHandle_t、hipsolverSpHandle_t、hipsolverRfHandle_t 和 hipsolverMgHandle_t。用户在调用任何其他库函数之前需要通过调用特定模块 Handle 的 Create 函数创建句柄对象，并作为参数传入其他库函数。

3.2 简单 demo

1) 稠密矩阵 LU 分解求解线性方程组

```
#include <cstdlib>
#include <hip/hip_runtime_api.h> // for hip functions
#include <hipsolver/hipsolver.h> // for all the hipsolver C interfaces and type declarations
#include <hip/library_types.h>

void print_matrix(const int &m, const int &n, const double *A, const int &lda)
{
    for (int i = 0; i < m; i++)
    {
```

```
            for (int j = 0; j < n; j++)
            {
                std::printf("%0.5lf ", A[j * lda + i]);
            }
            std::printf("\n");
        }
        return;
}

int main(int argc, char *argv[])
{
        const int m = 3;
        const int n = 3;
        const int lda = m;
        const int ldb = m;

        hipsolverHandle_t hipsolverH = NULL;
        hipStream_t stream = NULL;

        /*            | 1 2 3    |
         *      A = | 4 5 6    |
         *            | 7 8 10 |
         *
         * without pivoting: A = L*U
         *            | 1 0 0 |          | 1    2    3 |
         *      L = | 4 1 0 |, U = | 0 -3 -6 |
         *            | 7 2 1 |          | 0    0    1 |
         *
         * with pivoting: P*A = L*U
```

```
 *            | 0 0 1 |
 *      P = | 1 0 0 |
 *            | 0 1 0 |
 *
 *            | 1            0        0 |           | 7    8            10         |
 *      L = | 0.1429    1         0 |, U = | 0    0.8571    1.5714 |
 *            | 0.5714    0.5      1 |           | 0    0            -0.5      |
 */
double A[] = {1., 4., 7.,
                  2., 5., 8.,
                  3., 6., 10.};
double *X    = new double[m];
double *LU = new double[m*m];
int *h_devIpiv = new int[m];
int info = 0;

double *d_A = nullptr; /* device copy of A */
double *d_B = nullptr; /* device copy of B */
int *d_Ipiv = nullptr; /* pivoting sequence */
int *d_info = nullptr; /* error info */

int d_lwork = 0;                /* size of workspace */
double *d_work = nullptr; /* device workspace */
int h_lwork = 0;                /* size of workspace */
double *h_work = nullptr; /* host workspace */

print_matrix(m, n, A, lda);
std::printf("=====\n");
```

```
    /* step 1: create hipsolver handle, bind a stream */
    hipsolverDnCreate(&hipsolverH);

    hipStreamCreateWithFlags(&stream, hipStreamNonBlocking);
    hipsolverDnSetStream(hipsolverH, stream);

    /* step 2: copy A to device */
    hipMalloc(reinterpret_cast<void **>(&d_A), sizeof(double) * n * lda);
    hipMalloc(reinterpret_cast<void **>(&d_B), sizeof(double) * m);
    hipMalloc(reinterpret_cast<void **>(&d_Ipiv), sizeof(int) * m);
    hipMalloc(reinterpret_cast<void **>(&d_info), sizeof(int));

    hipMemcpyAsync(d_A, A, sizeof(double) * n * lda, hipMemcpyHostToDevice,
stream);

    /* step 3: query working space of getrf */
    hipsolverDgetrf_bufferSize(hipsolverH, m, n, d_A, lda, &d_lwork);

    hipMalloc(reinterpret_cast<void **>(&d_work), sizeof(double) * d_lwork);

    /* step 4: LU factorization */
    hipsolverDgetrf(hipsolverH, m, n, d_A, lda, d_work, d_lwork, d_Ipiv, d_info);
    hipMemcpyAsync(h_devIpiv, d_Ipiv, sizeof(int) * m, hipMemcpyDeviceToHost,
stream);
    hipMemcpyAsync(LU, d_A, sizeof(double) * m * m, hipMemcpyDeviceToHost,
stream);
    hipMemcpyAsync(&info, d_info, sizeof(int), hipMemcpyDeviceToHost, stream);

    hipStreamSynchronize(stream);
```

```
    std::printf("after Xgetrf: info = %d\n", info);
    if (0 > info)
    {
        std::printf("%d-th parameter is wrong \n", -info);
        exit(1);
    }
    std::printf("pivoting sequence, matlab base-1\n");
    for (int j = 0; j < m; j++)
    {
        std::printf("Ipiv(%d) = %d\n", j + 1, h_devIpiv[j]);
    }
    std::printf("L and U = (matlab base-1)\n");
    print_matrix(m, m, LU, lda);
    std::printf("=====\n");

    /*
     * step 5: solve A*X = B
     *            | 1 |            | -0.3333 |
     *     B = | 2 |,    X = |    0.6667 |
     *            | 3 |            |    0          |
     *
     */
    double B[] = {1., 2., 3.};
    print_matrix(m, 1, B, ldb);
    std::printf("=====\n");

    hipMemcpyAsync(d_B, B, sizeof(double) * m, hipMemcpyHostToDevice,
stream);
```

```
    hipsolverDgetrs(hipsolverH, HIPSOLVER_OP_N, m, 1, /* nrhs */
                                d_A, lda, (int *)d_Ipiv, d_B, ldb, d_work, d_lwork, d_info);
    hipMemcpyAsync(X, d_B, sizeof(double) * m, hipMemcpyDeviceToHost,
stream);
    hipStreamSynchronize(stream);

    std::printf("X = (matlab base-1)\n");
    print_matrix(m, 1, X, ldb);
    std::printf("=====\n");

    /* free resources */
    delete[] X;
    delete[] LU;
    delete[] h_devIpiv;
    hipFree(d_A);
    hipFree(d_B);
    hipFree(d_info);
    hipFree(d_Ipiv);
    hipFree(d_work);

    hipsolverDnDestroy(hipsolverH);
    hipStreamDestroy(stream);
    hipDeviceReset();

    return 0;
}
```

hipcc test_getrf_getri.cpp -o test_getrf_getri -lhipsolver

./ test_getrf_getri

4 API 详解

4.1 hipSolverDn

4.1.1 hipsolverDnCreate

```
hipsolverStatus_t hipsolverDnCreate(hipsolverHandle_t* handle);
/* This function initializes the hipSolverDn library and creates a handle on the hipSolverDn context.
handle - The pointer to the handle to the hipSolverDn context.
*/
```

4.1.2 hipsolverDnDestroy

```
hipsolverStatus_t hipsolverDnDestroy(hipsolverHandle_t handle);
/* This function releases CPU-side resources used by the    hipsolverDN library.
handle - The pointer to the handle to the hipSolverDn context.
*/
```

4.1.3 hipsolverDnSetStream

```
hipsolverStatus_t hipsolverDnSetStream(hipsolverHandle_t handle, hipStream_t streamId);
/* This function sets the stream to be used by the    hipsolverDN library to execute its routines.
handle - The pointer to the handle to the hipSolverDn context.
streamId - The stream to be used by the library.
*/
```

4.1.4 hipsolverDnGetStream

```
hipsolverStatus_t hipsolverDnGetStream(hipsolverHandle_t handle, hipStream_t* streamId);
/* This function queries the stream to be used by the    hipsolverDN library to execute its routines
```

```
handle - The pointer to the handle to the hipSolverDn context.
streamId - The stream which is used by handle.
*/
```

4.1.5 hipsolverDn<t>orgbr()

```
hipsolverStatus_t
hipsolverDnSorgbr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
      int m,
      int n,
      int k,
      const float *A,
      int lda,
      const float *tau,
      int *lwork);

hipsolverStatus_t
hipsolverDnDorgbr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
      int m,
      int n,
      int k,
      const double *A,
      int lda,
      const double *tau,
      int *lwork);

hipsolverStatus_t
```

```
hipsolverDnCungbr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
      int m,
      int n,
      int k,
      const    hipFloatComplex *A,
      int lda,
      const    hipFloatComplex *tau,
      int *lwork);

hipsolverStatus_t
hipsolverDnZungbr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
      int m,
      int n,
      int k,
      const    hipDoubleComplex *A,
      int lda,
      const    hipDoubleComplex *tau,
      int *lwork);

hipsolverStatus_t
hipsolverDnSorgbr(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
      int m,
      int n,
```

```
    int k,
    float *A,
    int lda,
    const float *tau,
    float *work,
    int lwork,
    int *devInfo);

hipsolverStatus_t
hipsolverDnDorgbr(
      hipsolverDnHandle_t handle,
      hipblasSideMode_t side,
    int m,
    int n,
    int k,
    double *A,
    int lda,
    const double *tau,
    double *work,
    int lwork,
    int *devInfo);
     hipsolverStatus_t
     hipsolverDnCungbr(
           hipsolverDnHandle_t handle,
           hipblasSideMode_t side,
          int m,
          int n,
          int k,
           hipFloatComplex *A,
```

```
            int lda,
            const    hipFloatComplex *tau,
             hipFloatComplex *work,
            int lwork,
            int *devInfo);


        hipsolverStatus_t
        hipsolverDnZungbr(
              hipsolverDnHandle_t handle,
              hipblasSideMode_t side,
            int m,
            int n,
            int k,
              hipDoubleComplex *A,
            int lda,
            const    hipDoubleComplex *tau,
              hipDoubleComplex *work,
            int lwork,
            int *devInfo);
/*
These helper functions calculate the size of work buffers needed.
This function generates one of the unitary matrices Q or P**H determined by gebrd
when reducing a matrix A to bidiagonal form:  Q^H * A * P = B
*/
```

4.1.6 hipsolverDn<t>orgqr()

```
 hipsolverStatus_t
 hipsolverDnSorgqr_bufferSize(
       hipsolverDnHandle_t handle,
      int m,
```

```
    int n,
    int k,
    const float *A,
    int lda,
    const float *tau,
    int *lwork);

hipsolverStatus_t
hipsolverDnDorgqr_bufferSize(
      hipsolverDnHandle_t handle,
    int m,
    int n,
    int k,
    const double *A,
    int lda,
    const double *tau,
    int *lwork);

hipsolverStatus_t
hipsolverDnCungqr_bufferSize(
      hipsolverDnHandle_t handle,
    int m,
    int n,
    int k,
    const    hipFloatComplex *A,
    int lda,
    const    hipFloatComplex *tau,
    int *lwork);
```

```
hipsolverStatus_t
hipsolverDnZungqr_bufferSize(
      hipsolverDnHandle_t handle,
    int m,
    int n,
    int k,
    const    hipDoubleComplex *A,
    int lda,
    const    hipDoubleComplex *tau,
    int *lwork);
     hipsolverStatus_t
     hipsolverDnSorgqr(
           hipsolverDnHandle_t handle,
          int m,
          int n,
          int k,
          float *A,
          int lda,
          const float *tau,
          float *work,
          int lwork,
          int *devInfo);

     hipsolverStatus_t
     hipsolverDnDorgqr(
           hipsolverDnHandle_t handle,
          int m,
          int n,
          int k,
```

```
        double *A,
        int lda,
        const double *tau,
        double *work,
        int lwork,
        int *devInfo);
    hipsolverStatus_t
    hipsolverDnCungqr(
          hipsolverDnHandle_t handle,
        int m,
        int n,
        int k,
          hipFloatComplex *A,
        int lda,
        const    hipFloatComplex *tau,
          hipFloatComplex *work,
        int lwork,
        int *devInfo);

    hipsolverStatus_t
    hipsolverDnZungqr(
          hipsolverDnHandle_t handle,
        int m,
        int n,
        int k,
          hipDoubleComplex *A,
        int lda,
        const    hipDoubleComplex *tau,
          hipDoubleComplex *work,
```

```
            int lwork,
            int *devInfo);
    /*
    */
```

4.1.7 hipsolverDn<t>orgtr()

```
hipsolverStatus_t
hipsolverDnSorgtr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *tau,
    int *lwork);

hipsolverStatus_t
hipsolverDnDorgtr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *tau,
    int *lwork);

hipsolverStatus_t
hipsolverDnCungtr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
```

```
    int n,
    const    hipFloatComplex *A,
    int lda,
    const    hipFloatComplex *tau,
    int *lwork);

hipsolverStatus_t
hipsolverDnZungtr_bufferSize(
      hipsolverDnHandle_t handle,
      hipblasFillMode_t uplo,
    int n,
    const    hipDoubleComplex *A,
    int lda,
    const    hipDoubleComplex *tau,
    int *lwork);
      hipsolverStatus_t
      hipsolverDnSorgtr(
            hipsolverDnHandle_t handle,
            hipblasFillMode_t uplo,
          int n,
          float *A,
          int lda,
          const float *tau,
          float *work,
          int lwork,
          int *devInfo);

      hipsolverStatus_t
      hipsolverDnDorgtr(
```

```
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    const double *tau,
    double *work,
    int lwork,
    int *devInfo);
hipsolverStatus_t
hipsolverDnCungtr(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
    int n,
        hipFloatComplex *A,
    int lda,
    const   hipFloatComplex *tau,
        hipFloatComplex *work,
    int lwork,
    int *devInfo);

hipsolverStatus_t
hipsolverDnZungtr(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
    int n,
        hipDoubleComplex *A,
    int lda,
    const   hipDoubleComplex *tau,
```

```
                hipDoubleComplex *work,
                int lwork,
                int *devInfo);
/* This function generates a unitary matrix Q which is defined as the product of n-1 elementary reflectors of order n, as returned by sytrd:
The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by orgtr_bufferSize(). Please note that the size in bytes of the working space is equal to sizeof(<type>) * lwork.
If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).
*/
```

4.1.8 hipsolverDn<t>ormqr()

```
hipsolverStatus_t
hipsolverDnSormqr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
        hipblasOperation_t trans,
      int m,
      int n,
      int k,
      const float *A,
      int lda,
      const float *tau,
      const float *C,
      int ldc,
      int *lwork);


hipsolverStatus_t
```

```
hipsolverDnDormqr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
        hipblasOperation_t trans,
      int m,
      int n,
      int k,
      const double *A,
      int lda,
      const double *tau,
      const double *C,
      int ldc,
      int *lwork);

hipsolverStatus_t
hipsolverDnCunmqr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
        hipblasOperation_t trans,
      int m,
      int n,
      int k,
      const    hipFloatComplex *A,
      int lda,
      const    hipFloatComplex *tau,
      const    hipFloatComplex *C,
      int ldc,
      int *lwork);
```

```
hipsolverStatus_t
hipsolverDnZunmqr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
        hipblasOperation_t trans,
      int m,
      int n,
      int k,
      const    hipDoubleComplex *A,
      int lda,
      const    hipDoubleComplex *tau,
      const    hipDoubleComplex *C,
      int ldc,
      int *lwork);
       hipsolverStatus_t
       hipsolverDnSormqr(
              hipsolverDnHandle_t handle,
              hipblasSideMode_t side,
              hipblasOperation_t trans,
            int m,
            int n,
            int k,
            const float *A,
            int lda,
            const float *tau,
            float *C,
            int ldc,
            float *work,
            int lwork,
```

```
        int *devInfo);

    hipsolverStatus_t
    hipsolverDnDormqr(
          hipsolverDnHandle_t handle,
          hipblasSideMode_t side,
          hipblasOperation_t trans,
        int m,
        int n,
        int k,
        const double *A,
        int lda,
        const double *tau,
        double *C,
        int ldc,
        double *work,
        int lwork,
        int *devInfo);
    hipsolverStatus_t
    hipsolverDnCunmqr(
          hipsolverDnHandle_t handle,
          hipblasSideMode_t side,
          hipblasOperation_t trans,
        int m,
        int n,
        int k,
        const    hipFloatComplex *A,
        int lda,
        const    hipFloatComplex *tau,
```

```
            hipFloatComplex *C,
            int ldc,
            hipFloatComplex *work,
            int lwork,
            int *devInfo);

        hipsolverStatus_t
        hipsolverDnZunmqr(
            hipsolverDnHandle_t handle,
            hipblasSideMode_t side,
            hipblasOperation_t trans,
            int m,
            int n,
            int k,
            const    hipDoubleComplex *A,
            int lda,
            const    hipDoubleComplex *tau,
            hipDoubleComplex *C,
            int ldc,
            hipDoubleComplex *work,
            int lwork,
            int *devInfo);
/*
```

The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by geqrf_bufferSize() or ormqr_bufferSize(). Please note that the size in bytes of the working space is equal to sizeof(<type>) * lwork.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).

```
The user can combine geqrf, ormqr and trsm to complete a linear solver or a least-square solver.
*/
```

4.1.9 hipsolverDn<t>ormtr()

```
hipsolverStatus_t
hipsolverDnSormtr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
        hipblasFillMode_t uplo,
        hipblasOperation_t trans,
      int m,
      int n,
      const float *A,
      int lda,
      const float *tau,
      const float *C,
      int ldc,
      int *lwork);

hipsolverStatus_t
hipsolverDnDormtr_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
        hipblasFillMode_t uplo,
        hipblasOperation_t trans,
      int m,
      int n,
      const double *A,
      int lda,
```

```
    const double *tau,
    const double *C,
    int ldc,
    int *lwork);

hipsolverStatus_t
hipsolverDnCunmtr_bufferSize(
      hipsolverDnHandle_t handle,
      hipblasSideMode_t side,
      hipblasFillMode_t uplo,
      hipblasOperation_t trans,
    int m,
    int n,
    const    hipFloatComplex *A,
    int lda,
    const    hipFloatComplex *tau,
    const    hipFloatComplex *C,
    int ldc,
    int *lwork);

hipsolverStatus_t
hipsolverDnZunmtr_bufferSize(
      hipsolverDnHandle_t handle,
      hipblasSideMode_t side,
      hipblasFillMode_t uplo,
      hipblasOperation_t trans,
    int m,
    int n,
    const    hipDoubleComplex *A,
```

```
int lda,
const    hipDoubleComplex *tau,
const    hipDoubleComplex *C,
int ldc,
int *lwork);
 hipsolverStatus_t
 hipsolverDnSormtr(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
        hipblasFillMode_t uplo,
        hipblasOperation_t trans,
       int m,
       int n,
       float *A,
       int lda,
       float *tau,
       float *C,
       int ldc,
       float *work,
       int lwork,
       int *devInfo);

 hipsolverStatus_t
 hipsolverDnDormtr(
        hipsolverDnHandle_t handle,
        hipblasSideMode_t side,
        hipblasFillMode_t uplo,
        hipblasOperation_t trans,
       int m,
```

```
        int n,
        double *A,
        int lda,
        double *tau,
        double *C,
        int ldc,
        double *work,
        int lwork,
        int *devInfo);
    hipsolverStatus_t
    hipsolverDnCunmtr(
          hipsolverDnHandle_t handle,
          hipblasSideMode_t side,
          hipblasFillMode_t uplo,
          hipblasOperation_t trans,
        int m,
        int n,
          hipFloatComplex *A,
        int lda,
          hipFloatComplex *tau,
          hipFloatComplex *C,
        int ldc,
          hipFloatComplex *work,
        int lwork,
        int *devInfo);

    hipsolverStatus_t
    hipsolverDnZunmtr(
          hipsolverDnHandle_t handle,
```

```
                hipblasSideMode_t side,
                hipblasFillMode_t uplo,
                hipblasOperation_t trans,
               int m,
               int n,
                hipDoubleComplex *A,
               int lda,
                hipDoubleComplex *tau,
                hipDoubleComplex *C,
               int ldc,
                hipDoubleComplex *work,
               int lwork,
               int *devInfo);
/*
The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by ormtr_bufferSize(). Please note that the size in bytes of the working space is equal to sizeof(<type>) * lwork.
If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).
*/
```

4.1.10 hipsolverDn<t>gebrd()

```
 hipsolverStatus_t
 hipsolverDnSgebrd_bufferSize(
        hipsolverDnHandle_t handle,
       int m,
       int n,
       int *Lwork );
```

```
hipsolverStatus_t
hipsolverDnDgebrd_bufferSize(
        hipsolverDnHandle_t handle,
      int m,
      int n,
      int *Lwork );

hipsolverStatus_t
hipsolverDnCgebrd_bufferSize(
        hipsolverDnHandle_t handle,
      int m,
      int n,
      int *Lwork );

hipsolverStatus_t
hipsolverDnZgebrd_bufferSize(
        hipsolverDnHandle_t handle,
      int m,
      int n,
      int *Lwork );
       hipsolverStatus_t
       hipsolverDnSgebrd( hipsolverDnHandle_t handle,
                          int m,
                          int n,
                          float *A,
                          int lda,
                          float *D,
                          float *E,
                          float *TAUQ,
```

```
                float *TAUP,
                float *Work,
                int Lwork,
                int *devInfo );

hipsolverStatus_t
hipsolverDnDgebrd( hipsolverDnHandle_t handle,
                int m,
                int n,
                double *A,
                int lda,
                double *D,
                double *E,
                double *TAUQ,
                double *TAUP,
                double *Work,
                int Lwork,
                int *devInfo );
hipsolverStatus_t
hipsolverDnCgebrd( hipsolverDnHandle_t handle,
                int m,
                int n,
                 hipFloatComplex *A,
                int lda,
                float *D,
                float *E,
                 hipFloatComplex *TAUQ,
                 hipFloatComplex *TAUP,
                 hipFloatComplex *Work,
```

```
                    int Lwork,
                    int *devInfo );

    hipsolverStatus_t
    hipsolverDnZgebrd( hipsolverDnHandle_t handle,
                    int m,
                    int n,
                     hipDoubleComplex *A,
                    int lda,
                    double *D,
                    double *E,
                     hipDoubleComplex *TAUQ,
                     hipDoubleComplex *TAUP,
                     hipDoubleComplex *Work,
                    int Lwork,
                    int *devInfo );
/*
This function reduces a general  m × n  matrix A to a real upper or lower bidiagonal
form B by an orthogonal transform:  Q^H * A * P = B
*/
```

4.1.11 hipsolverDn<t><t>gels()

```
 hipsolverStatus_t
 hipsolverDnZZgels_bufferSize(
      hipsolverHandle_t                  handle,
    int                                  m,
    int                                  n,
    int                                  nrhs,
     hipDoubleComplex                 *     dA,
    int                                  ldda,
```

```
    hipDoubleComplex                *     dB,
    int                               lddb,
    hipDoubleComplex                *     dX,
    int                               lddx,
    void                            *     dwork,
    size_t                          *     lwork_bytes);

hipsolverStatus_t
hipsolverDnZCgels_bufferSize(
    hipsolverHandle_t                 handle,
    int                               m,
    int                               n,
    int                               nrhs,
    hipDoubleComplex                *     dA,
    int                               ldda,
    hipDoubleComplex                *     dB,
    int                               lddb,
    hipDoubleComplex                *     dX,
    int                               lddx,
    void                            *     dwork,
    size_t                          *     lwork_bytes);

hipsolverStatus_t
hipsolverDnZKgels_bufferSize(
    hipsolverHandle_t                 handle,
    int                               m,
    int                               n,
    int                               nrhs,
    hipDoubleComplex                *     dA,
```

```
    int                                 ldda,
     hipDoubleComplex                *      dB,
    int                                 lddb,
     hipDoubleComplex                *      dX,
    int                                 lddx,
    void                             *      dwork,
    size_t                          *      lwork_bytes);

hipsolverStatus_t
hipsolverDnZEgels_bufferSize(
     hipsolverHandle_t                  handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
     hipDoubleComplex                *      dA,
    int                                 ldda,
     hipDoubleComplex                *      dB,
    int                                 lddb,
     hipDoubleComplex                *      dX,
    int                                 lddx,
    void                             *      dwork,
    size_t                          *      lwork_bytes);

hipsolverStatus_t
hipsolverDnZYgels_bufferSize(
     hipsolverHandle_t                  handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
```

```
    hipDoubleComplex                * dA,
    int                               ldda,
    hipDoubleComplex                * dB,
    int                               lddb,
    hipDoubleComplex                * dX,
    int                               lddx,
    void                            * dwork,
    size_t                          * lwork_bytes);

hipsolverStatus_t
hipsolverDnCCgels_bufferSize(
    hipsolverHandle_t                 handle,
    int                               m,
    int                               n,
    int                               nrhs,
    hipFloatComplex                     * dA,
    int                               ldda,
    hipFloatComplex                     * dB,
    int                               lddb,
    hipFloatComplex                     * dX,
    int                               lddx,
    void                            * dwork,
    size_t                          * lwork_bytes);

hipsolverStatus_t
hipsolverDnCKgels_bufferSize(
    hipsolverHandle_t                 handle,
    int                               m,
    int                               n,
```

```
    int                                 nrhs,
     hipFloatComplex                      *     dA,
    int                                 ldda,
     hipFloatComplex                      *     dB,
    int                                 lddb,
     hipFloatComplex                      *     dX,
    int                                 lddx,
    void                              *     dwork,
    size_t                          *     lwork_bytes);

hipsolverStatus_t
hipsolverDnCEgels_bufferSize(
     hipsolverHandle_t                    handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
     hipFloatComplex                      *     dA,
    int                                 ldda,
     hipFloatComplex                      *     dB,
    int                                 lddb,
     hipFloatComplex                      *     dX,
    int                                 lddx,
    void                              *     dwork,
    size_t                          *     lwork_bytes);

hipsolverStatus_t
hipsolverDnCYgels_bufferSize(
     hipsolverHandle_t                    handle,
    int                                 m,
```

```
    int                                 n,
    int                                 nrhs,
     hipFloatComplex                      *     dA,
    int                                 ldda,
     hipFloatComplex                      *     dB,
    int                                 lddb,
     hipFloatComplex                      *     dX,
    int                                 lddx,
    void                          *     dwork,
    size_t                        *     lwork_bytes);

hipsolverStatus_t
hipsolverDnDDgels_bufferSize(
     hipsolverHandle_t                   handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
    double                        *     dA,
    int                                 ldda,
    double                        *     dB,
    int                                 lddb,
    double                        *     dX,
    int                                 lddx,
    void                          *     dwork,
    size_t                        *     lwork_bytes);

hipsolverStatus_t
hipsolverDnDSgels_bufferSize(
     hipsolverHandle_t                   handle,
```

```
    int                                 m,
    int                                 n,
    int                                 nrhs,
    double                        *     dA,
    int                                 ldda,
    double                        *     dB,
    int                                 lddb,
    double                        *     dX,
    int                                 lddx,
    void                          *     dwork,
    size_t                        *     lwork_bytes);

hipsolverStatus_t
hipsolverDnDHgels_bufferSize(
     hipsolverHandle_t                   handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
    double                        *     dA,
    int                                 ldda,
    double                        *     dB,
    int                                 lddb,
    double                        *     dX,
    int                                 lddx,
    void                          *     dwork,
    size_t                        *     lwork_bytes);

hipsolverStatus_t
hipsolverDnDBgels_bufferSize(
```

```
    hipsolverHandle_t                handle,
    int                              m,
    int                              n,
    int                              nrhs,
    double                 *         dA,
    int                              ldda,
    double                 *         dB,
    int                              lddb,
    double                 *         dX,
    int                              lddx,
    void                   *         dwork,
    size_t                 *         lwork_bytes);

hipsolverStatus_t
hipsolverDnDXgels_bufferSize(
    hipsolverHandle_t                handle,
    int                              m,
    int                              n,
    int                              nrhs,
    double                 *         dA,
    int                              ldda,
    double                 *         dB,
    int                              lddb,
    double                 *         dX,
    int                              lddx,
    void                   *         dwork,
    size_t                 *         lwork_bytes);

hipsolverStatus_t
```

```
hipsolverDnSSgels_bufferSize(
      hipsolverHandle_t                 handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
    float                         *     dA,
    int                                 ldda,
    float                         *     dB,
    int                                 lddb,
    float                         *     dX,
    int                                 lddx,
    void                           *    dwork,
    size_t                        *     lwork_bytes);

hipsolverStatus_t
hipsolverDnSHgels_bufferSize(
      hipsolverHandle_t                 handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
    float                         *     dA,
    int                                 ldda,
    float                         *     dB,
    int                                 lddb,
    float                         *     dX,
    int                                 lddx,
    void                           *    dwork,
    size_t                        *     lwork_bytes);
```

```
hipsolverStatus_t
hipsolverDnSBgels_bufferSize(
      hipsolverHandle_t                 handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
    float                         *     dA,
    int                                 ldda,
    float                         *     dB,
    int                                 lddb,
    float                         *     dX,
    int                                 lddx,
    void                           *    dwork,
    size_t                        *     lwork_bytes);

hipsolverStatus_t
hipsolverDnSXgels_bufferSize(
      hipsolverHandle_t                 handle,
    int                                 m,
    int                                 n,
    int                                 nrhs,
    float                         *     dA,
    int                                 ldda,
    float                         *     dB,
    int                                 lddb,
    float                         *     dX,
    int                                 lddx,
    void                           *    dwork,
    size_t                        *     lwork_bytes);
```

/*

These functions compute the solution of a system of linear equations with one or multiple right hand sides using mixed precision iterative refinement techniques based on the QR factorization Xgels. These functions are similar in term of functionalities to the full precision LAPACK QR (least squares) solver (Xgels, where X denotes Z,C,D,S) but it uses lower precision internally in order to provide faster time to solution, from here comes the name mixed precision. Mixed precision iterative refinement techniques means that the solver compute an QR factorization in lower precision and then iteratively refine the solution to achieve the accuracy of the Inputs/Outputs datatype precision. The <t1> corresponds to the Inputs/Outputs datatype precision while <t2> represent the internal lower precision at which the factorization will be carried on.

$$A \times X = B$$

Where A is m-by-n matrix and X is n-by-nrhs and B is m-by-nrhs matrices.

*/

4.1.12 hipsolverDn<t><t>gesv()

```
hipsolverStatus_t
hipsolverDnZZgesv_bufferSize(
      hipsolverHandle_t                handle,
    int                                n,
    int                                nrhs,
     hipDoubleComplex                *     dA,
    int                                lda,
    int                              *     dipiv,
     hipDoubleComplex                *     dB,
    int                                lddb,
     hipDoubleComplex                *     dX,
    int                                lddx,
    void                              *     dwork,
    size_t                           *     lwork_bytes);
```

```
hipsolverStatus_t
hipsolverDnZCgesv_bufferSize(
      hipsolverHandle_t                handle,
    int                                n,
    int                                nrhs,
      hipDoubleComplex               *     dA,
    int                                lda,
    int                              *     dipiv,
      hipDoubleComplex               *     dB,
    int                                lddb,
      hipDoubleComplex               *     dX,
    int                                lddx,
    void                             *     dwork,
    size_t                           *     lwork_bytes);

hipsolverStatus_t
hipsolverDnZKgesv_bufferSize(
      hipsolverHandle_t                handle,
    int                                n,
    int                                nrhs,
      hipDoubleComplex               *     dA,
    int                                lda,
    int                              *     dipiv,
      hipDoubleComplex               *     dB,
    int                                lddb,
      hipDoubleComplex               *     dX,
    int                                lddx,
    void                             *     dwork,
```

```
    size_t                          *     lwork_bytes);

hipsolverStatus_t
hipsolverDnZEgesv_bufferSize(
     hipsolverHandle_t                 handle,
    int                                n,
    int                                nrhs,
     hipDoubleComplex                *     dA,
    int                                lda,
    int                             *     dipiv,
     hipDoubleComplex                *     dB,
    int                                lddb,
     hipDoubleComplex                *     dX,
    int                                lddx,
    void                             *     dwork,
    size_t                          *     lwork_bytes);

hipsolverStatus_t
hipsolverDnZYgesv_bufferSize(
     hipsolverHandle_t                 handle,
    int                                n,
    int                                nrhs,
     hipDoubleComplex                *     dA,
    int                                lda,
    int                             *     dipiv,
     hipDoubleComplex                *     dB,
    int                                lddb,
     hipDoubleComplex                *     dX,
    int                                lddx,
```

```
    void                            *    dwork,
    size_t                          *    lwork_bytes);

hipsolverStatus_t
hipsolverDnCCgesv_bufferSize(
     hipsolverHandle_t                   handle,
    int                                  n,
    int                                  nrhs,
     hipFloatComplex                     *    dA,
    int                                  ldda,
    int                             *    dipiv,
     hipFloatComplex                     *    dB,
    int                                  lddb,
     hipFloatComplex                     *    dX,
    int                                  lddx,
    void                            *    dwork,
    size_t                          *    lwork_bytes);

hipsolverStatus_t
hipsolverDnCKgesv_bufferSize(
     hipsolverHandle_t                   handle,
    int                                  n,
    int                                  nrhs,
     hipFloatComplex                     *    dA,
    int                                  ldda,
    int                             *    dipiv,
     hipFloatComplex                     *    dB,
    int                                  lddb,
     hipFloatComplex                     *    dX,
```

```
    int                                 lddx,
    void                          *     dwork,
    size_t                        *     lwork_bytes);

hipsolverStatus_t
hipsolverDnCEgesv_bufferSize(
     hipsolverHandle_t                   handle,
    int                                 n,
    int                                 nrhs,
     hipFloatComplex                         *     dA,
    int                                 ldda,
    int                           *     dipiv,
     hipFloatComplex                         *     dB,
    int                                 lddb,
     hipFloatComplex                         *     dX,
    int                                 lddx,
    void                          *     dwork,
    size_t                        *     lwork_bytes);

hipsolverStatus_t
hipsolverDnCYgesv_bufferSize(
     hipsolverHandle_t                   handle,
    int                                 n,
    int                                 nrhs,
     hipFloatComplex                         *     dA,
    int                                 ldda,
    int                           *     dipiv,
     hipFloatComplex                         *     dB,
    int                                 lddb,
```

```
    hipFloatComplex                 *     dX,
    int                                   lddx,
    void                            *     dwork,
    size_t                          *     lwork_bytes);

hipsolverStatus_t
hipsolverDnDDgesv_bufferSize(
    hipsolverHandle_t                     handle,
    int                                   n,
    int                                   nrhs,
    double                          *     dA,
    int                                   ldda,
    int                             *     dipiv,
    double                          *     dB,
    int                                   lddb,
    double                          *     dX,
    int                                   lddx,
    void                            *     dwork,
    size_t                          *     lwork_bytes);

hipsolverStatus_t
hipsolverDnDSgesv_bufferSize(
    hipsolverHandle_t                     handle,
    int                                   n,
    int                                   nrhs,
    double                          *     dA,
    int                                   ldda,
    int                             *     dipiv,
    double                          *     dB,
```

```
    int                                 lddb,
    double                  *           dX,
    int                                 lddx,
    void                    *           dwork,
    size_t                  *           lwork_bytes);

hipsolverStatus_t
hipsolverDnDHgesv_bufferSize(
      hipsolverHandle_t                  handle,
    int                                 n,
    int                                 nrhs,
    double                  *           dA,
    int                                 ldda,
    int                     *           dipiv,
    double                  *           dB,
    int                                 lddb,
    double                  *           dX,
    int                                 lddx,
    void                    *           dwork,
    size_t                  *           lwork_bytes);

hipsolverStatus_t
hipsolverDnDBgesv_bufferSize(
      hipsolverHandle_t                  handle,
    int                                 n,
    int                                 nrhs,
    double                  *           dA,
    int                                 ldda,
    int                     *           dipiv,
```

```
    double                          *   dB,
    int                                 lddb,
    double                          *   dX,
    int                                 lddx,
    void                            *   dwork,
    size_t                          *   lwork_bytes);

hipsolverStatus_t
hipsolverDnDXgesv_bufferSize(
     hipsolverHandle_t                  handle,
    int                                 n,
    int                                 nrhs,
    double                          *   dA,
    int                                 ldda,
    int                             *   dipiv,
    double                          *   dB,
    int                                 lddb,
    double                          *   dX,
    int                                 lddx,
    void                            *   dwork,
    size_t                          *   lwork_bytes);

hipsolverStatus_t
hipsolverDnSSgesv_bufferSize(
     hipsolverHandle_t                  handle,
    int                                 n,
    int                                 nrhs,
    float                           *   dA,
    int                                 ldda,
```

```
    int                             *   dipiv,
    float                           *   dB,
    int                                 lddb,
    float                           *   dX,
    int                                 lddx,
    void                            *   dwork,
    size_t                          *   lwork_bytes);

hipsolverStatus_t
hipsolverDnSHgesv_bufferSize(
     hipsolverHandle_t                  handle,
    int                                 n,
    int                                 nrhs,
    float                           *   dA,
    int                                 ldda,
    int                             *   dipiv,
    float                           *   dB,
    int                                 lddb,
    float                           *   dX,
    int                                 lddx,
    void                            *   dwork,
    size_t                          *   lwork_bytes);

hipsolverStatus_t
hipsolverDnSBgesv_bufferSize(
     hipsolverHandle_t                  handle,
    int                                 n,
    int                                 nrhs,
    float                           *   dA,
```

```
    int                                 ldda,
    int                             *   dipiv,
    float                           *   dB,
    int                                 lddb,
    float                           *   dX,
    int                                 lddx,
    void                            *   dwork,
    size_t                          *   lwork_bytes);

hipsolverStatus_t
hipsolverDnSXgesv_bufferSize(
     hipsolverHandle_t                  handle,
    int                                 n,
    int                                 nrhs,
    float                           *   dA,
    int                                 ldda,
    int                             *   dipiv,
    float                           *   dB,
    int                                 lddb,
    float                           *   dX,
    int                                 lddx,
    void                            *   dwork,
    size_t                          *   lwork_bytes);
/*
These functions are modelled after functions DSGESV and ZCGESV from LAPACK.
They compute the solution of a system of linear equations with one or multiple right
hand sides using mixed precision iterative refinement techniques based on the LU
factorization Xgesv. These functions are similar in term of functionalities to the full
precision LU solver (Xgesv, where X denotes Z,C,D,S) but it uses lower precision
```

internally in order to provide faster time to solution, from here comes the name mixed precision. Mixed precision iterative refinement techniques means that the solver compute an LU factorization in lower precision and then iteratively refine the solution to achieve the accuracy of the Inputs/Outputs datatype precision. The <t1> corresponds to the Inputs/Outputs datatype precision while <t2> represent the internal lower precision at which the factorization will be carried on.

$$A \times X = B$$

Where A is n-by-n matrix and X and B are n-by-nrhs matrices.

*/

4.1.13 hipsolverDn<t>geqrf()

```
hipsolverStatus_t
hipsolverDnSgeqrf_bufferSize( hipsolverDnHandle_t handle,
                              int m,
                              int n,
                              float *A,
                              int lda,
                              int *Lwork );

hipsolverStatus_t
hipsolverDnDgeqrf_bufferSize( hipsolverDnHandle_t handle,
                              int m,
                              int n,
                              double *A,
                              int lda,
                              int *Lwork );

hipsolverStatus_t
hipsolverDnCgeqrf_bufferSize( hipsolverDnHandle_t handle,
                              int m,
```

```
                                int n,
                                 hipFloatComplex *A,
                                int lda,
                                int *Lwork );

hipsolverStatus_t
hipsolverDnZgeqrf_bufferSize( hipsolverDnHandle_t handle,
                                int m,
                                int n,
                                 hipDoubleComplex *A,
                                int lda,
                                int *Lwork );
hipsolverStatus_t
hipsolverDnSgeqrf( hipsolverDnHandle_t handle,
                int m,
                int n,
                float *A,
                int lda,
                float *TAU,
                float *Workspace,
                int Lwork,
                int *devInfo );

hipsolverStatus_t
hipsolverDnDgeqrf( hipsolverDnHandle_t handle,
                int m,
                int n,
                double *A,
                int lda,
```

```
                double *TAU,
                double *Workspace,
                int Lwork,
                int *devInfo );
 hipsolverStatus_t
 hipsolverDnCgeqrf( hipsolverDnHandle_t handle,
                int m,
                int n,
                  hipFloatComplex *A,
                int lda,
                  hipFloatComplex *TAU,
                  hipFloatComplex *Workspace,
                int Lwork,
                int *devInfo );

 hipsolverStatus_t
 hipsolverDnZgeqrf( hipsolverDnHandle_t handle,
                int m,
                int n,
                  hipDoubleComplex *A,
                int lda,
                  hipDoubleComplex *TAU,
                  hipDoubleComplex *Workspace,
                int Lwork,
                int *devInfo );
/*
This function computes the QR factorization of an matrix
                                    A = Q * R
where A is an matrix, Q is an matrix, and R is a upper triangular matrix.
```

```
*/
```

4.1.14 hipsolverDn<t>getrf()

```
hipsolverStatus_t
hipsolverDnSgetrf_bufferSize( hipsolverDnHandle_t handle,
                              int m,
                              int n,
                              float *A,
                              int lda,
                              int *Lwork );

hipsolverStatus_t
hipsolverDnDgetrf_bufferSize( hipsolverDnHandle_t handle,
                              int m,
                              int n,
                              double *A,
                              int lda,
                              int *Lwork );

hipsolverStatus_t
hipsolverDnCgetrf_bufferSize( hipsolverDnHandle_t handle,
                              int m,
                              int n,
                                hipFloatComplex *A,
                              int lda,
                              int *Lwork );

hipsolverStatus_t
hipsolverDnZgetrf_bufferSize( hipsolverDnHandle_t handle,
                              int m,
```

```
                                int n,
                                 hipDoubleComplex *A,
                                int lda,
                                int *Lwork );
hipsolverStatus_t
hipsolverDnSgetrf( hipsolverDnHandle_t handle,
                int m,
                int n,
                float *A,
                int lda,
                float *Workspace,
                int *devIpiv,
                int *devInfo );

hipsolverStatus_t
hipsolverDnDgetrf( hipsolverDnHandle_t handle,
                int m,
                int n,
                double *A,
                int lda,
                double *Workspace,
                int *devIpiv,
                int *devInfo );
hipsolverStatus_t
hipsolverDnCgetrf( hipsolverDnHandle_t handle,
                int m,
                int n,
                 hipFloatComplex *A,
                int lda,
```

```
                  hipFloatComplex *Workspace,
                int *devIpiv,
                int *devInfo );

 hipsolverStatus_t
 hipsolverDnZgetrf( hipsolverDnHandle_t handle,
                int m,
                int n,
                  hipDoubleComplex *A,
                int lda,
                  hipDoubleComplex *Workspace,
                int *devIpiv,
                int *devInfo );
/*
This function computes the LU factorization of an matrix
                                  P * A = L * U
where A is an matrix, P is a permutation matrix, L is a lower triangular matrix with unit
diagonal, and U is an upper triangular matrix.
The user has to provide working space which is pointed by input parameter Workspace.
The input parameter Lwork is size of the working space, and it is returned by
getrf_bufferSize().
*/
```

4.1.15 hipsolverDn<t>getrs()

```
 hipsolverStatus_t
 hipsolverDnSgetrs( hipsolverDnHandle_t handle,
                  hipblasOperation_t trans,
                int n,
                int nrhs,
                const float *A,
```

```
                int lda,
                const int *devIpiv,
                float *B,
                int ldb,
                int *devInfo );

hipsolverStatus_t
hipsolverDnDgetrs( hipsolverDnHandle_t handle,
                  hipblasOperation_t trans,
                int n,
                int nrhs,
                const double *A,
                int lda,
                const int *devIpiv,
                double *B,
                int ldb,
                int *devInfo );

hipsolverStatus_t
hipsolverDnCgetrs( hipsolverDnHandle_t handle,
                  hipblasOperation_t trans,
                int n,
                int nrhs,
                const    hipFloatComplex *A,
                int lda,
                const int *devIpiv,
                  hipFloatComplex *B,
                int ldb,
                int *devInfo );
```

```
hipsolverStatus_t
hipsolverDnZgetrs( hipsolverDnHandle_t handle,
                   hipblasOperation_t trans,
                  int n,
                  int nrhs,
                  const    hipDoubleComplex *A,
                  int lda,
                  const int *devIpiv,
                   hipDoubleComplex *B,
                  int ldb,
                  int *devInfo );
/*
This function solves a linear system of multiple right-hand sides
                                    op(A) * X = B
where A is an matrix, and was LU-factored by getrf, that is, lower triangular part of A
is L, and upper triangular part (including diagonal elements) of A is U. B is an right-
hand side matrix.
*/
```

4.1.16 hipsolverDn<t>potrf()

```
hipsolverStatus_t
hipsolverDnSpotrf_bufferSize( hipsolverDnHandle_t handle,
                              hipblasFillMode_t uplo,
                             int n,
                             float *A,
                             int lda,
                             int *Lwork );


hipsolverStatus_t
```

```
hipsolverDnDpotrf_bufferSize( hipsolverDnHandle_t handle,
                              hipblasFillMode_t uplo,
                             int n,
                             double *A,
                             int lda,
                             int *Lwork );

hipsolverStatus_t
hipsolverDnCpotrf_bufferSize( hipsolverDnHandle_t handle,
                              hipblasFillMode_t uplo,
                             int n,
                              hipFloatComplex *A,
                             int lda,
                             int *Lwork );

hipsolverStatus_t
hipsolverDnZpotrf_bufferSize( hipsolverDnHandle_t handle,
                              hipblasFillMode_t uplo,
                             int n,
                              hipDoubleComplex *A,
                             int lda,
                             int *Lwork);
hipsolverStatus_t
hipsolverDnSpotrf( hipsolverDnHandle_t handle,
                    hipblasFillMode_t uplo,
                   int n,
                   float *A,
                   int lda,
                   float *Workspace,
```

```
                int Lwork,
                int *devInfo );

hipsolverStatus_t
hipsolverDnDpotrf( hipsolverDnHandle_t handle,
                  hipblasFillMode_t uplo,
                int n,
                double *A,
                int lda,
                double *Workspace,
                int Lwork,
                int *devInfo );
hipsolverStatus_t
hipsolverDnCpotrf( hipsolverDnHandle_t handle,
                  hipblasFillMode_t uplo,
                int n,
                  hipFloatComplex *A,
                int lda,
                  hipFloatComplex *Workspace,
                int Lwork,
                int *devInfo );

hipsolverStatus_t
hipsolverDnZpotrf( hipsolverDnHandle_t handle,
                  hipblasFillMode_t uplo,
                int n,
                  hipDoubleComplex *A,
                int lda,
                  hipDoubleComplex *Workspace,
```

```
                int Lwork,
                int *devInfo );
/*
This function computes the Cholesky factorization of a Hermitian positive-definite matrix. A is an Hermitian matrix, only the lower or upper part is meaningful. The input parameter uplo indicates which part of the matrix is used. The function will leave the other part untouched.
*/
```

4.1.17 hipsolverDn<t>potrfBatched()

```
hipsolverStatus_t
hipsolverDnSpotrfBatched(
      hipsolverDnHandle_t handle,
      hipblasFillMode_t uplo,
     int n,
     float *Aarray[],
     int lda,
     int *infoArray,
     int batchSize);

hipsolverStatus_t
hipsolverDnDpotrfBatched(
      hipsolverDnHandle_t handle,
      hipblasFillMode_t uplo,
     int n,
     double *Aarray[],
     int lda,
     int *infoArray,
     int batchSize);
      hipsolverStatus_t
```

```
        hipsolverDnCpotrfBatched(
              hipsolverDnHandle_t handle,
              hipblasFillMode_t uplo,
             int n,
              hipFloatComplex *Aarray[],
             int lda,
             int *infoArray,
             int batchSize);


        hipsolverStatus_t
        hipsolverDnZpotrfBatched(
              hipsolverDnHandle_t handle,
              hipblasFillMode_t uplo,
             int n,
              hipDoubleComplex *Aarray[],
             int lda,
             int *infoArray,
             int batchSize);
/*
This function computes the Cholesky factorization of a sequence of Hermitian positive-
definite matrices.
*/
```

4.1.18 hipsolverDn<t>potrs()

```
 hipsolverStatus_t
 hipsolverDnSpotrs( hipsolverDnHandle_t handle,
                    hipblasFillMode_t uplo,
                   int n,
                   int nrhs,
                   const float *A,
```

```
                int lda,
                float *B,
                int ldb,
                int *devInfo);

hipsolverStatus_t
hipsolverDnDpotrs( hipsolverDnHandle_t handle,
                  hipblasFillMode_t uplo,
                int n,
                int nrhs,
                const double *A,
                int lda,
                double *B,
                int ldb,
                int *devInfo);

hipsolverStatus_t
hipsolverDnCpotrs( hipsolverDnHandle_t handle,
                  hipblasFillMode_t uplo,
                int n,
                int nrhs,
                const    hipFloatComplex *A,
                int lda,
                  hipFloatComplex *B,
                int ldb,
                int *devInfo);

hipsolverStatus_t
hipsolverDnZpotrs( hipsolverDnHandle_t handle,
```

```
                  hipblasFillMode_t uplo,
                int n,
                int nrhs,
                const    hipDoubleComplex *A,
                int lda,
                  hipDoubleComplex *B,
                int ldb,
                int *devInfo);
/*
This function solves a system of linear equations
                                        A * X = B
where A is an Hermitian matrix, only lower or upper part is meaningful. The input parameter uplo indicates which part of the matrix is used. The function will leave the other part untouched.
*/
```

4.1.19 hipsolverDn<t>potrsBatched()

```
 hipsolverStatus_t
 hipsolverDnSpotrsBatched(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
       int n,
       int nrhs,
       float *Aarray[],
       int lda,
       float *Barray[],
       int ldb,
       int *info,
       int batchSize);
```

```
hipsolverStatus_t
hipsolverDnDpotrsBatched(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
      int n,
      int nrhs,
      double *Aarray[],
      int lda,
      double *Barray[],
      int ldb,
      int *info,
      int batchSize);

hipsolverStatus_t
hipsolverDnCpotrsBatched(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
      int n,
      int nrhs,
        hipFloatComplex *Aarray[],
      int lda,
        hipFloatComplex *Barray[],
      int ldb,
      int *info,
      int batchSize);

hipsolverStatus_t
hipsolverDnZpotrsBatched(
        hipsolverDnHandle_t handle,
```

```
        hipblasFillMode_t uplo,
       int n,
       int nrhs,
        hipDoubleComplex *Aarray[],
       int lda,
        hipDoubleComplex *Barray[],
       int ldb,
       int *info,
       int batchSize);
/*
This function solves a sequence of linear systems
                                A[i] * X[i] = B[i]
where each Aarray[i] for i=0,1,..., batchSize-1 is a Hermitian matrix, only lower or upper part is meaningful. The input parameter uplo indicates which part of the matrix is used.
*/
```

4.1.20 hipsolverDn<t>potri()

```
 hipsolverStatus_t
 hipsolverDnSpotri_bufferSize( hipsolverDnHandle_t handle,
                          hipblasFillMode_t uplo,
                         int n,
                         float *A,
                         int lda,
                         int *Lwork );

 hipsolverStatus_t
 hipsolverDnDpotri_bufferSize( hipsolverDnHandle_t handle,
                          hipblasFillMode_t uplo,
```

```
                        int n,
                        double *A,
                        int lda,
                        int *Lwork );

hipsolverStatus_t
hipsolverDnCpotri_bufferSize( hipsolverDnHandle_t handle,
                          hipblasFillMode_t uplo,
                        int n,
                          hipFloatComplex *A,
                        int lda,
                        int *Lwork );

hipsolverStatus_t
hipsolverDnZpotri_bufferSize( hipsolverDnHandle_t handle,
                          hipblasFillMode_t uplo,
                        int n,
                          hipDoubleComplex *A,
                        int lda,
                        int *Lwork);
hipsolverStatus_t
hipsolverDnSpotri( hipsolverDnHandle_t handle,
                  hipblasFillMode_t uplo,
                int n,
                float *A,
                int lda,
                float *Workspace,
                int Lwork,
                int *devInfo );
```

```
hipsolverStatus_t
hipsolverDnDpotri( hipsolverDnHandle_t handle,
                   hipblasFillMode_t uplo,
                   int n,
                   double *A,
                   int lda,
                   double *Workspace,
                   int Lwork,
                   int *devInfo );
hipsolverStatus_t
hipsolverDnCpotri( hipsolverDnHandle_t handle,
                   hipblasFillMode_t uplo,
                   int n,
                   hipFloatComplex *A,
                   int lda,
                   hipFloatComplex *Workspace,
                   int Lwork,
                   int *devInfo );

hipsolverStatus_t
hipsolverDnZpotri( hipsolverDnHandle_t handle,
                   hipblasFillMode_t uplo,
                   int n,
                   hipDoubleComplex *A,
                   int lda,
                   hipDoubleComplex *Workspace,
                   int Lwork,
                   int *devInfo );
```

/*

This function computes the inverse of a positive-definite matrix A using the Cholesky factorization

$$A = L * L^H = U^H * U$$

computed by potrf().

*/

4.1.21 hipsolverDn<t>gesvd()

```
hipsolverStatus_t
hipsolverDnSgesvd_bufferSize(
        hipsolverDnHandle_t handle,
      int m,
      int n,
      int *lwork );

hipsolverStatus_t
hipsolverDnDgesvd_bufferSize(
        hipsolverDnHandle_t handle,
      int m,
      int n,
      int *lwork );

hipsolverStatus_t
hipsolverDnCgesvd_bufferSize(
        hipsolverDnHandle_t handle,
      int m,
      int n,
      int *lwork );

hipsolverStatus_t
```

```
hipsolverDnZgesvd_bufferSize(
        hipsolverDnHandle_t handle,
      int m,
      int n,
      int *lwork );
        hipsolverStatus_t
        hipsolverDnSgesvd (
              hipsolverDnHandle_t handle,
            signed char jobu,
            signed char jobvt,
            int m,
            int n,
            float *A,
            int lda,
            float *S,
            float *U,
            int ldu,
            float *VT,
            int ldvt,
            float *work,
            int lwork,
            float *rwork,
            int *devInfo);

        hipsolverStatus_t
        hipsolverDnDgesvd (
              hipsolverDnHandle_t handle,
            signed char jobu,
            signed char jobvt,
```

```
        int m,
        int n,
        double *A,
        int lda,
        double *S,
        double *U,
        int ldu,
        double *VT,
        int ldvt,
        double *work,
        int lwork,
        double *rwork,
        int *devInfo);
    hipsolverStatus_t
    hipsolverDnCgesvd (
          hipsolverDnHandle_t handle,
        signed char jobu,
        signed char jobvt,
        int m,
        int n,
          hipFloatComplex *A,
        int lda,
        float *S,
          hipFloatComplex *U,
        int ldu,
          hipFloatComplex *VT,
        int ldvt,
          hipFloatComplex *work,
        int lwork,
```

```
        float *rwork,
        int *devInfo);

    hipsolverStatus_t
    hipsolverDnZgesvd (
          hipsolverDnHandle_t handle,
        signed char jobu,
        signed char jobvt,
        int m,
        int n,
          hipDoubleComplex *A,
        int lda,
        double *S,
          hipDoubleComplex *U,
        int ldu,
          hipDoubleComplex *VT,
        int ldvt,
          hipDoubleComplex *work,
        int lwork,
        double *rwork,
        int *devInfo);
/*
This function computes the singular value decomposition (SVD) of an m × n matrix
A and corresponding the left and/or right singular vectors. The SVD is written
                              A = U ∗ Σ ∗ V^H
where Σ is an m × n matrix which is zero except for its min(m,n) diagonal elements,
U is an m × n unitary matrix, and V is an m × n unitary matrix.
*/
```

4.1.22 hipsolverDn<t>gesvdj()

```
hipsolverStatus_t
hipsolverDnSgesvdj_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
      int econ,
      int m,
      int n,
      const float *A,
      int lda,
      const float *S,
      const float *U,
      int ldu,
      const float *V,
      int ldv,
      int *lwork,
      gesvdjInfo_t params);

hipsolverStatus_t
hipsolverDnDgesvdj_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
      int econ,
      int m,
      int n,
      const double *A,
      int lda,
      const double *S,
      const double *U,
```

```
    int ldu,
    const double *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params);

hipsolverStatus_t
hipsolverDnCgesvdj_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    const    hipFloatComplex *A,
    int lda,
    const float *S,
    const    hipFloatComplex *U,
    int ldu,
    const    hipFloatComplex *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params);

hipsolverStatus_t
hipsolverDnZgesvdj_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
    int econ,
    int m,
```

```
    int n,
    const    hipDoubleComplex *A,
    int lda,
    const double *S,
    const    hipDoubleComplex *U,
    int ldu,
    const    hipDoubleComplex *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params);
hipsolverStatus_t
hipsolverDnSgesvdj(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    float *A,
    int lda,
    float *S,
    float *U,
    int ldu,
    float *V,
    int ldv,
    float *work,
    int lwork,
    int *info,
    gesvdjInfo_t params);
```

```
hipsolverStatus_t
hipsolverDnDgesvdj(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
      int econ,
      int m,
      int n,
      double *A,
      int lda,
      double *S,
      double *U,
      int ldu,
      double *V,
      int ldv,
      double *work,
      int lwork,
      int *info,
      gesvdjInfo_t params);
       hipsolverStatus_t
       hipsolverDnCgesvdj(
              hipsolverDnHandle_t handle,
              hipsolverEigMode_t jobz,
            int econ,
            int m,
            int n,
              hipFloatComplex *A,
            int lda,
            float *S,
              hipFloatComplex *U,
```

```
        int ldu,
         hipFloatComplex *V,
        int ldv,
         hipFloatComplex *work,
        int lwork,
        int *info,
        gesvdjInfo_t params);

    hipsolverStatus_t
    hipsolverDnZgesvdj(
         hipsolverDnHandle_t handle,
         hipsolverEigMode_t jobz,
        int econ,
        int m,
        int n,
         hipDoubleComplex *A,
        int lda,
        double *S,
         hipDoubleComplex *U,
        int ldu,
         hipDoubleComplex *V,
        int ldv,
         hipDoubleComplex *work,
        int lwork,
        int *info,
        gesvdjInfo_t params);
/*
This function computes the singular value decomposition (SVD) of an m × n matrix
A and corresponding the left and/or right singular vectors. The SVD is written:
```

$$A = U * \Sigma * V^H$$

where Σ is an $m \times n$ matrix which is zero except for its min(m,n) diagonal elements, U is an $m \times n$ unitary matrix, and V is an $m \times n$ unitary matrix.

*/

4.1.23 hipsolverDn<t>gesvdjBatched()

```
hipsolverStatus_t
hipsolverDnSgesvdjBatched_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
      int m,
      int n,
      const float *A,
      int lda,
      const float *S,
      const float *U,
      int ldu,
      const float *V,
      int ldv,
      int *lwork,
      gesvdjInfo_t params,
      int batchSize);

hipsolverStatus_t
hipsolverDnDgesvdjBatched_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
      int m,
      int n,
      const double *A,
```

```
    int lda,
    const double *S,
    const double *U,
    int ldu,
    const double *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params,
    int batchSize);

hipsolverStatus_t
hipsolverDnCgesvdjBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
    int m,
    int n,
    const    hipFloatComplex *A,
    int lda,
    const float *S,
    const    hipFloatComplex *U,
    int ldu,
    const    hipFloatComplex *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params,
    int batchSize);

hipsolverStatus_t
hipsolverDnZgesvdjBatched_bufferSize(
```

```
  hipsolverDnHandle_t handle,
  hipsolverEigMode_t jobz,
int m,
int n,
const    hipDoubleComplex *A,
int lda,
const double *S,
const    hipDoubleComplex *U,
int ldu,
const    hipDoubleComplex *V,
int ldv,
int *lwork,
gesvdjInfo_t params,
int batchSize);
  hipsolverStatus_t
  hipsolverDnSgesvdjBatched(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
        int m,
        int n,
        float *A,
        int lda,
        float *S,
        float *U,
        int ldu,
        float *V,
        int ldv,
        float *work,
        int lwork,
```

```
        int *info,
        gesvdjInfo_t params,
        int batchSize);

hipsolverStatus_t
hipsolverDnDgesvdjBatched(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
        int m,
        int n,
        double *A,
        int lda,
        double *S,
        double *U,
        int ldu,
        double *V,
        int ldv,
        double *work,
        int lwork,
        int *info,
        gesvdjInfo_t params,
        int batchSize);
hipsolverStatus_t
hipsolverDnCgesvdjBatched(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
        int m,
        int n,
          hipFloatComplex *A,
```

```
        int lda,
        float *S,
         hipFloatComplex *U,
        int ldu,
         hipFloatComplex *V,
        int ldv,
         hipFloatComplex *work,
        int lwork,
        int *info,
        gesvdjInfo_t params,
        int batchSize);

hipsolverStatus_t
hipsolverDnZgesvdjBatched(
         hipsolverDnHandle_t handle,
         hipsolverEigMode_t jobz,
        int m,
        int n,
         hipDoubleComplex *A,
        int lda,
        double *S,
         hipDoubleComplex *U,
        int ldu,
         hipDoubleComplex *V,
        int ldv,
         hipDoubleComplex *work,
        int lwork,
        int *info,
        gesvdjInfo_t params,
```

```
            int batchSize);
/*
This function computes singular values and singular vectors of a sequence of general
m × n matrices
                    A_j = U_j * Σ_j * V_j^H
where Σ_j is a real m × n diagonal matrix which is zero except for its min(m,n)
diagonal elements. U_j (left singular vectors) is an m × m unitary matrix and V_j
(right singular vectors) is a n × n unitary matrix. The diagonal elements of A_j are the
singular values of in either descending order or non-sorting order.
*/
```

4.1.24 hipsolverDn<t>gesvdaStridedBatched()

```
hipsolverStatus_t
hipsolverDnSgesvdaStridedBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const float *A,
    int lda,
    long long int strideA,
    const float *S,
    long long int strideS,
    const float *U,
    int ldu,
    long long int strideU,
    const float *V,
    int ldv,
```

```
    long long int strideV,
    int *lwork,
    int batchSize);

hipsolverStatus_t
hipsolverDnDgesvdaStridedBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const double *A,
    int lda,
    long long int strideA,
    const double *S,
    long long int strideS,
    const double *U,
    int ldu,
    long long int strideU,
    const double *V,
    int ldv,
    long long int strideV,
    int *lwork,
    int batchSize);

hipsolverStatus_t
hipsolverDnCgesvdaStridedBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
```

```
    int rank,
    int m,
    int n,
    const    hipFloatComplex *A,
    int lda,
    long long int strideA,
    const float *S,
    long long int strideS,
    const    hipFloatComplex *U,
    int ldu,
    long long int strideU,
    const    hipFloatComplex *V,
    int ldv,
    long long int strideV,
    int *lwork,
    int batchSize);

hipsolverStatus_t
hipsolverDnZgesvdaStridedBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const    hipDoubleComplex *A,
    int lda,
    long long int strideA,
    const double *S,
    long long int strideS,
```

```
const    hipDoubleComplex *U,
int ldu,
long long int strideU,
const    hipDoubleComplex *V,
int ldv,
long long int strideV,
int *lwork,
int batchSize);
 hipsolverStatus_t
 hipsolverDnSgesvdaStridedBatched(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
       int rank,
       int m,
       int n,
       const float *A,
       int lda,
       long long int strideA,
       float *S,
       long long int strideS,
       float *U,
       int ldu,
       long long int strideU,
       float *V,
       int ldv,
       long long int strideV,
       float *work,
       int lwork,
       int *info,
```

```
        double *h_R_nrmF,
        int batchSize);

    hipsolverStatus_t
    hipsolverDnDgesvdaStridedBatched(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
        int rank,
        int m,
        int n,
        const double *A,
        int lda,
        long long int strideA,
        double *S,
        long long int strideS,
        double *U,
        int ldu,
        long long int strideU,
        double *V,
        int ldv,
        long long int strideV,
        double *work,
        int lwork,
        int *info,
        double *h_R_nrmF,
        int batchSize);
    hipsolverStatus_t
    hipsolverDnCgesvdaStridedBatched(
          hipsolverDnHandle_t handle,
```

```
        hipsolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const    hipFloatComplex *A,
    int lda,
    long long int strideA,
    float *S,
    long long int strideS,
     hipFloatComplex *U,
    int ldu,
    long long int strideU,
     hipFloatComplex *V,
    int ldv,
    long long int strideV,
     hipFloatComplex *work,
    int lwork,
    int *info,
    double *h_R_nrmF,
    int batchSize);

hipsolverStatus_t
hipsolverDnZgesvdaStridedBatched(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const    hipDoubleComplex *A,
```

```
        int lda,
        long long int strideA,
        double *S,
        long long int strideS,
         hipDoubleComplex *U,
        int ldu,
        long long int strideU,
         hipDoubleComplex *V,
        int ldv,
        long long int strideV,
         hipDoubleComplex *work,
        int lwork,
        int *info,
        double *h_R_nrmF,
        int batchSize);
/*
This function gesvda (a stands for approximate) approximates the singular value decomposition of a tall skinny m × n matrix A and corresponding the left and right singular vectors. The economy form of SVD is written by
                                A = U ∗ Σ ∗ V^H
where Σ is an n × n matrix. U is an m × m unitary matrix, and V is an m × n unitary matrix. The diagonal elements of Σ are the singular values of A; they are real and non-negative, and are returned in descending order. U and V are the left and right singular vectors of A.
*/
```

4.1.25 hipsolverDn<t>syevd()

```
hipsolverStatus_t
hipsolverDnSsyevd_bufferSize(
      hipsolverDnHandle_t handle,
```

```
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
      const float *A,
      int lda,
      const float *W,
      int *lwork);

hipsolverStatus_t
hipsolverDnDsyevd_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
      const double *A,
      int lda,
      const double *W,
      int *lwork);

hipsolverStatus_t
hipsolverDnCheevd_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
      const    hipFloatComplex *A,
      int lda,
      const float *W,
      int *lwork);
```

```
hipsolverStatus_t
hipsolverDnZheevd_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
      const    hipDoubleComplex *A,
      int lda,
      const double *W,
      int *lwork);
       hipsolverStatus_t
       hipsolverDnSsyevd(
              hipsolverDnHandle_t handle,
              hipsolverEigMode_t jobz,
              hipblasFillMode_t uplo,
            int n,
            float *A,
            int lda,
            float *W,
            float *work,
            int lwork,
            int *devInfo);

       hipsolverStatus_t
       hipsolverDnDsyevd(
              hipsolverDnHandle_t handle,
              hipsolverEigMode_t jobz,
              hipblasFillMode_t uplo,
```

```
        int n,
        double *A,
        int lda,
        double *W,
        double *work,
        int lwork,
        int *devInfo);
    hipsolverStatus_t
    hipsolverDnCheevd(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
          hipblasFillMode_t uplo,
        int n,
          hipFloatComplex *A,
        int lda,
        float *W,
          hipFloatComplex *work,
        int lwork,
        int *devInfo);

    hipsolverStatus_t
    hipsolverDnZheevd(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
          hipblasFillMode_t uplo,
        int n,
          hipDoubleComplex *A,
        int lda,
        double *W,
```

```
                hipDoubleComplex *work,
                int lwork,
                int *devInfo);
/*
This function computes eigenvalues and eigenvectors of a symmetric (Hermitian)
 n × n  matrix A. The standard symmetric eigenvalue problem is
                                A ∗ V = V ∗ Λ
where Λ is a real  n × n  diagonal matrix. V is an  n × n  unitary matrix. The diagonal
elements of Λ are the eigenvalues of A in ascending order.
*/
```

4.1.26 hipsolverDn<t>syevdx()

```
 hipsolverStatus_t
 hipsolverDnSsyevdx_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
       int n,
       const float *A,
       int lda,
       float vl,
       float vu,
       int il,
       int iu,
       int *h_meig,
       const float *W,
       int *lwork);


 hipsolverStatus_t
```

```
hipsolverDnDsyevdx_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
      int n,
      const double *A,
      int lda,
      double vl,
      double vu,
      int il,
      int iu,
      int *h_meig,
      const double *W,
      int *lwork);

hipsolverStatus_t
hipsolverDnCheevdx_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
      int n,
      const    hipFloatComplex *A,
      int lda,
      float vl,
      float vu,
      int il,
      int iu,
```

```
    int *h_meig,
    const float *W,
    int *lwork);

hipsolverStatus_t
hipsolverDnZheevdx_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
      hipsolverEigRange_t range,
      hipblasFillMode_t uplo,
    int n,
    const    hipDoubleComplex *A,
    int lda,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
    const double *W,
    int *lwork);
     hipsolverStatus_t
     hipsolverDnSsyevdx(
           hipsolverDnHandle_t handle,
           hipsolverEigMode_t jobz,
           hipsolverEigRange_t range,
           hipblasFillMode_t uplo,
          int n,
          float *A,
          int lda,
```

```
        float vl,
        float vu,
        int il,
        int iu,
        int *h_meig,
        float *W,
        float *work,
        int lwork,
        int *devInfo);

    hipsolverStatus_t
    hipsolverDnDsyevdx(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
          hipsolverEigRange_t range,
          hipblasFillMode_t uplo,
        int n,
        double *A,
        int lda,
        double vl,
        double vu,
        int il,
        int iu,
        int *h_meig,
        double *W,
        double *work,
        int lwork,
        int *devInfo);
    hipsolverStatus_t
```

```
hipsolverDnCheevdx(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
      int n,
        hipFloatComplex *A,
      int lda,
      float vl,
      float vu,
      int il,
      int iu,
      int *h_meig,
      float *W,
        hipFloatComplex *work,
      int lwork,
      int *devInfo);

hipsolverStatus_t
hipsolverDnZheevdx(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
      int n,
        hipDoubleComplex *A,
      int lda,
      double vl,
      double vu,
```

```
            int il,
            int iu,
            int *h_meig,
            double *W,
             hipDoubleComplex *work,
            int lwork,
            int *devInfo);
/*
This function computes all or selection of the eigenvalues and optionally eigenvectors of a symmetric (Hermitian) n×n matrix A. The standard symmetric eigenvalue problem is:
                                A∗V=V∗Λ
where Λ is a real n×h_meig diagonal matrix. V is an n×h_meig unitary matrix.
*/
```

4.1.27 hipsolverDn<t>syevj()

```
hipsolverStatus_t
hipsolverDnSsyevj_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
     int n,
     const float *A,
     int lda,
     const float *W,
     int *lwork,
     syevjInfo_t params);

hipsolverStatus_t
hipsolverDnDsyevj_bufferSize(
```

```
    hipsolverDnHandle_t handle,
    hipsolverEigMode_t jobz,
    hipblasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *W,
    int *lwork,
    syevjInfo_t params);

hipsolverStatus_t
hipsolverDnCheevj_bufferSize(
    hipsolverDnHandle_t handle,
    hipsolverEigMode_t jobz,
    hipblasFillMode_t uplo,
    int n,
    const    hipFloatComplex *A,
    int lda,
    const float *W,
    int *lwork,
    syevjInfo_t params);

hipsolverStatus_t
hipsolverDnZheevj_bufferSize(
    hipsolverDnHandle_t handle,
    hipsolverEigMode_t jobz,
    hipblasFillMode_t uplo,
    int n,
    const    hipDoubleComplex *A,
```

```
int lda,
const double *W,
int *lwork,
syevjInfo_t params);
  hipsolverStatus_t
  hipsolverDnSsyevj(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
       int n,
       float *A,
       int lda,
       float *W,
       float *work,
       int lwork,
       int *info,
       syevjInfo_t params);

  hipsolverStatus_t
  hipsolverDnDsyevj(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
       int n,
       double *A,
       int lda,
       double *W,
       double *work,
       int lwork,
```

```
        int *info,
        syevjInfo_t params);
    hipsolverStatus_t
    hipsolverDnCheevj(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
          hipblasFillMode_t uplo,
        int n,
          hipFloatComplex *A,
        int lda,
        float *W,
          hipFloatComplex *work,
        int lwork,
        int *info,
        syevjInfo_t params);

    hipsolverStatus_t
    hipsolverDnZheevj(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
          hipblasFillMode_t uplo,
        int n,
          hipDoubleComplex *A,
        int lda,
        double *W,
          hipDoubleComplex *work,
        int lwork,
        int *info,
        syevjInfo_t params);
```

/*

This function computes eigenvalues and eigenvectors of a symmetric (Hermitian) $n \times n$ matrix A. The standard symmetric eigenvalue problem is

$$A * Q = Q * \Lambda$$

where Λ is a real $n \times n$ diagonal matrix. Q is an $n \times n$ unitary matrix. The diagonal elements of Λ are the eigenvalues of A in ascending order.

*/

4.1.28 hipsolverDn<t>syevjBatched()

```
hipsolverStatus_t
hipsolverDnSsyevjBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *W,
    int *lwork,
    syevjInfo_t params,
    int batchSize
    );

hipsolverStatus_t
hipsolverDnDsyevjBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int n,
    const double *A,
```

```
    int lda,
    const double *W,
    int *lwork,
    syevjInfo_t params,
    int batchSize
    );

hipsolverStatus_t
hipsolverDnCheevjBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int n,
    const    hipFloatComplex *A,
    int lda,
    const float *W,
    int *lwork,
    syevjInfo_t params,
    int batchSize
    );

hipsolverStatus_t
hipsolverDnZheevjBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int n,
    const    hipDoubleComplex *A,
    int lda,
```

```
const double *W,
int *lwork,
syevjInfo_t params,
int batchSize
);
 hipsolverStatus_t
 hipsolverDnSsyevjBatched(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
       int n,
       float *A,
       int lda,
       float *W,
       float *work,
       int lwork,
       int *info,
       syevjInfo_t params,
       int batchSize
       );

 hipsolverStatus_t
 hipsolverDnDsyevjBatched(
        hipsolverDnHandle_t handle,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
       int n,
       double *A,
       int lda,
```

```
        double *W,
        double *work,
        int lwork,
        int *info,
        syevjInfo_t params,
        int batchSize
        );
    hipsolverStatus_t
    hipsolverDnCheevjBatched(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
          hipblasFillMode_t uplo,
        int n,
          hipFloatComplex *A,
        int lda,
        float *W,
          hipFloatComplex *work,
        int lwork,
        int *info,
        syevjInfo_t params,
        int batchSize
        );

    hipsolverStatus_t
    hipsolverDnZheevjBatched(
          hipsolverDnHandle_t handle,
          hipsolverEigMode_t jobz,
          hipblasFillMode_t uplo,
        int n,
```

```
            hipDoubleComplex *A,
            int lda,
            double *W,
            hipDoubleComplex *work,
            int lwork,
            int *info,
            syevjInfo_t params,
            int batchSize
            );
/*
This function computes eigenvalues and eigenvectors of a sequence of symmetric (Hermitian) n × n matrices
                                A_j * Q_j = Q_j * Λ_j
where Λ_j is a real n × n diagonal matrix. Q_j is an n × n unitary matrix. The diagonal elements of Λ_j are the eigenvalues of A_j in either ascending order or non-sorting order.
*/
```

4.1.29 hipsolverDnXsyevBatched()

```
hipsolverStatus_t
hipsolverDnXsyevBatched_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverDnParams_t params,
      hipsolverEigMode_t  jobz,
      hipblasFillMode_t uplo,
     int64_t n,
      hipDataType dataTypeA,
     const void *A,
     int64_t lda,
      hipDataType dataTypeW,
```

```
    const void *W,
      hipDataType computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost,
    int64_t batchSize)
      hipsolverStatus_t
      hipsolverDnXsyevBatched(
            hipsolverDnHandle_t handle,
            hipsolverDnParams_t params,
            hipsolverEigMode_t jobz,
            hipblasFillMode_t uplo,
          int64_t n,
            hipDataType dataTypeA,
          void *A,
          int64_t lda,
            hipDataType dataTypeW,
          void *W,
            hipDataType computeType,
          void *bufferOnDevice,
          size_t workspaceInBytesOnDevice,
          void *bufferOnHost,
          size_t workspaceInBytesOnHost,
          int *info,
          int64_t batchSize)
/*
```

The helper functions below can calculate the sizes needed for pre-allocated buffer. The following routine computes eigenvalues and eigenvectors of a sequence of symmetric (Hermitian) $n \times n$ matrices

$$A_j * V_j = V_j * \Lambda_j$$

```
where Λ_j is a real n × n diagonal matrix.
*/
```

4.1.30 hipsolverDn<t>sygvd()

```
hipsolverStatus_t
hipsolverDnSsygvd_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
      const float *A,
      int lda,
      const float *B,
      int ldb,
      const float *W,
      int *lwork);

hipsolverStatus_t
hipsolverDnDsygvd_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
      const double *A,
      int lda,
      const double *B,
      int ldb,
```

```
    const double *W,
    int *lwork);

hipsolverStatus_t
hipsolverDnChegvd_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigType_t itype,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int n,
    const    hipFloatComplex *A,
    int lda,
    const    hipFloatComplex *B,
    int ldb,
    const float *W,
    int *lwork);

hipsolverStatus_t
hipsolverDnZhegvd_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigType_t itype,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int n,
    const    hipDoubleComplex *A,
    int lda,
    const    hipDoubleComplex *B,
    int ldb,
    const double *W,
```

```
int *lwork);
 hipsolverStatus_t
 hipsolverDnSsygvd(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
       int n,
       float *A,
       int lda,
       float *B,
       int ldb,
       float *W,
       float *work,
       int lwork,
       int *devInfo);

 hipsolverStatus_t
 hipsolverDnDsygvd(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
       int n,
       double *A,
       int lda,
       double *B,
       int ldb,
       double *W,
```

```
        double *work,
        int lwork,
        int *devInfo);
    hipsolverStatus_t
    hipsolverDnChegvd(
          hipsolverDnHandle_t handle,
          hipsolverEigType_t itype,
          hipsolverEigMode_t jobz,
          hipblasFillMode_t uplo,
        int n,
          hipFloatComplex *A,
        int lda,
          hipFloatComplex *B,
        int ldb,
        float *W,
          hipFloatComplex *work,
        int lwork,
        int *devInfo);

    hipsolverStatus_t
    hipsolverDnZhegvd(
          hipsolverDnHandle_t handle,
          hipsolverEigType_t itype,
          hipsolverEigMode_t jobz,
          hipblasFillMode_t uplo,
        int n,
          hipDoubleComplex *A,
        int lda,
          hipDoubleComplex *B,
```

```
            int ldb,
            double *W,
             hipDoubleComplex *work,
            int lwork,
            int *devInfo);
/*
This function computes eigenvalues and eigenvectors of a symmetric (Hermitian) n × n matrix-pair (A,B).
*/
```

4.1.31 hipsolverDn<t>sygvdx()

```
hipsolverStatus_t
hipsolverDnSsygvdx_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
      int n,
      const float *A,
      int lda,
      const float *B,
      int ldb,
      float vl,
      float vu,
      int il,
      int iu,
      int *h_meig,
      const float *W,
      int *lwork);
```

```
hipsolverStatus_t
hipsolverDnDsygvdx_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
      int n,
      const double *A,
      int lda,
      const double *B,
      int ldb,
      double vl,
      double vu,
      int il,
      int iu,
      int *h_meig,
      const double *W,
      int *lwork);

hipsolverStatus_t
hipsolverDnChegvdx_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
      int n,
```

```
    const    hipFloatComplex *A,
    int lda,
    const    hipFloatComplex *B,
    int ldb,
    float vl,
    float vu,
    int il,
    int iu,
    int *h_meig,
    const float *W,
    int *lwork);

hipsolverStatus_t
hipsolverDnZhegvdx_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigType_t itype,
      hipsolverEigMode_t jobz,
      hipsolverEigRange_t range,
      hipblasFillMode_t uplo,
    int n,
    const    hipDoubleComplex *A,
    int lda,
    const    hipDoubleComplex *B,
    int ldb,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
```

```
    const double *W,
    int *lwork);
     hipsolverStatus_t
     hipsolverDnSsygvdx(
            hipsolverDnHandle_t handle,
            hipsolverEigType_t itype,
            hipsolverEigMode_t jobz,
            hipsolverEigRange_t range,
            hipblasFillMode_t uplo,
           int n,
           float *A,
           int lda,
           float *B,
           int ldb,
           float vl,
           float vu,
           int il,
           int iu,
           int *h_meig,
           float *W,
           float *work,
           int lwork,
           int *devInfo);

     hipsolverStatus_t
     hipsolverDnDsygvdx(
            hipsolverDnHandle_t handle,
            hipsolverEigType_t itype,
            hipsolverEigMode_t jobz,
```

```
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double *B,
    int ldb,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
    double *W,
    double *work,
    int lwork,
    int *devInfo);
hipsolverStatus_t
hipsolverDnChegvdx(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipsolverEigRange_t range,
        hipblasFillMode_t uplo,
    int n,
        hipFloatComplex *A,
    int lda,
        hipFloatComplex *B,
    int ldb,
    float vl,
```

```
        float vu,
        int il,
        int iu,
        int *h_meig,
        float *W,
         hipFloatComplex *work,
        int lwork,
        int *devInfo);

hipsolverStatus_t
hipsolverDnZhegvdx(
         hipsolverDnHandle_t handle,
         hipsolverEigType_t itype,
         hipsolverEigMode_t jobz,
         hipsolverEigRange_t range,
         hipblasFillMode_t uplo,
        int n,
         hipDoubleComplex *A,
        int lda,
         hipDoubleComplex *B,
        int ldb,
        double vl,
        double vu,
        int il,
        int iu,
        int *h_meig,
        double *W,
         hipDoubleComplex *work,
        int lwork,
```

```
            int *devInfo);
/*
This function computes all or selection of the eigenvalues and optionally eigenvectors
of a symmetric (Hermitian) n × n matrix-pair (A,B).
*/
```

4.1.32 hipsolverDn<t>sygvj()

```
hipsolverStatus_t
hipsolverDnSsygvj_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
      const float *A,
      int lda,
      const float *B,
      int ldb,
      const float *W,
      int *lwork,
      syevjInfo_t params);

hipsolverStatus_t
hipsolverDnDsygvj_bufferSize(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
      const double *A,
```

```
    int lda,
    const double *B,
    int ldb,
    const double *W,
    int *lwork,
    syevjInfo_t params);

hipsolverStatus_t
hipsolverDnChegvj_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigType_t itype,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int n,
    const    hipFloatComplex *A,
    int lda,
    const    hipFloatComplex *B,
    int ldb,
    const float *W,
    int *lwork,
    syevjInfo_t params);

hipsolverStatus_t
hipsolverDnZhegvj_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverEigType_t itype,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int n,
```

```
const    hipDoubleComplex *A,
int lda,
const    hipDoubleComplex *B,
int ldb,
const double *W,
int *lwork,
syevjInfo_t params);
 hipsolverStatus_t
 hipsolverDnSsygvj(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
       int n,
       float *A,
       int lda,
       float *B,
       int ldb,
       float *W,
       float *work,
       int lwork,
       int *info,
       syevjInfo_t params);

 hipsolverStatus_t
 hipsolverDnDsygvj(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
```

```
        hipblasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double *B,
    int ldb,
    double *W,
    double *work,
    int lwork,
    int *info,
    syevjInfo_t params);
hipsolverStatus_t
hipsolverDnChegvj(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
    int n,
        hipFloatComplex *A,
    int lda,
        hipFloatComplex *B,
    int ldb,
    float *W,
        hipFloatComplex *work,
    int lwork,
    int *info,
    syevjInfo_t params);

hipsolverStatus_t
```

```
hipsolverDnZhegvj(
        hipsolverDnHandle_t handle,
        hipsolverEigType_t itype,
        hipsolverEigMode_t jobz,
        hipblasFillMode_t uplo,
      int n,
        hipDoubleComplex *A,
      int lda,
        hipDoubleComplex *B,
      int ldb,
      double *W,
        hipDoubleComplex *work,
      int lwork,
      int *info,
      syevjInfo_t params);
/*
This function computes eigenvalues and eigenvectors of a symmetric (Hermitian) n × n matrix-pair (A,B).
*/
```

4.1.33 hipsolverDn<t>sytrd()

```
hipsolverStatus_t
hipsolverDnSsytrd_bufferSize(
      hipsolverDnHandle_t handle,
      hipblasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *d,
    const float *e,
```

```
    const float *tau,
    int *lwork);

hipsolverStatus_t
hipsolverDnDsytrd_bufferSize(
      hipsolverDnHandle_t handle,
      hipblasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *d,
    const double *e,
    const double *tau,
    int *lwork);

hipsolverStatus_t
hipsolverDnChetrd_bufferSize(
      hipsolverDnHandle_t handle,
      hipblasFillMode_t uplo,
    int n,
    const    hipFloatComplex *A,
    int lda,
    const float *d,
    const float *e,
    const    hipFloatComplex *tau,
    int *lwork);

hipsolverStatus_t
hipsolverDnZhetrd_bufferSize(
```

```
  hipsolverDnHandle_t handle,
  hipblasFillMode_t uplo,
int n,
const    hipDoubleComplex *A,
int lda,
const double *d,
const double *e,
const    hipDoubleComplex *tau,
int *lwork);
  hipsolverStatus_t
  hipsolverDnSsytrd(
         hipsolverDnHandle_t handle,
         hipblasFillMode_t uplo,
        int n,
        float *A,
        int lda,
        float *d,
        float *e,
        float *tau,
        float *work,
        int lwork,
        int *devInfo);

  hipsolverStatus_t
  hipsolverDnDsytrd(
         hipsolverDnHandle_t handle,
         hipblasFillMode_t uplo,
        int n,
        double *A,
```

```
        int lda,
        double *d,
        double *e,
        double *tau,
        double *work,
        int lwork,
        int *devInfo);
hipsolverStatus_t
hipsolverDnChetrd(
          hipsolverDnHandle_t handle,
          hipblasFillMode_t uplo,
        int n,
          hipFloatComplex *A,
        int lda,
        float *d,
        float *e,
          hipFloatComplex *tau,
          hipFloatComplex *work,
        int lwork,
        int *devInfo);

hipsolverStatus_t CUDENSEAPI    hipsolverDnZhetrd(
          hipsolverDnHandle_t handle,
          hipblasFillMode_t uplo,
        int n,
          hipDoubleComplex *A,
        int lda,
        double *d,
        double *e,
```

```
                hipDoubleComplex *tau,
                hipDoubleComplex *work,
               int lwork,
               int *devInfo);
/*
This function reduces a general symmetric (Hermitian) n × n matrix A to real
symmetric tridiagonal form T by an orthogonal transformation:
                                  Q^H * A * Q = T
*/
```

4.1.34 hipsolverDn<t>sytrf()

```
hipsolverStatus_t
hipsolverDnSsytrf_bufferSize( hipsolverDnHandle_t handle,
                              int n,
                              float *A,
                              int lda,
                              int *lwork );

hipsolverStatus_t
hipsolverDnDsytrf_bufferSize( hipsolverDnHandle_t handle,
                              int n,
                              double *A,
                              int lda,
                              int *lwork );

hipsolverStatus_t
hipsolverDnCsytrf_bufferSize( hipsolverDnHandle_t handle,
                              int n,
                               hipFloatComplex *A,
                              int lda,
```

```
                                int *lwork );

hipsolverStatus_t
hipsolverDnZsytrf_bufferSize( hipsolverDnHandle_t handle,
                                int n,
                                 hipDoubleComplex *A,
                                int lda,
                                int *lwork );
hipsolverStatus_t
hipsolverDnSsytrf( hipsolverDnHandle_t handle,
                  hipblasFillMode_t uplo,
                int n,
                float *A,
                int lda,
                int *devIpiv,
                float *work,
                int lwork,
                int *devInfo );

hipsolverStatus_t
hipsolverDnDsytrf( hipsolverDnHandle_t handle,
                  hipblasFillMode_t uplo,
                int n,
                double *A,
                int lda,
                int *devIpiv,
                double *work,
                int lwork,
                int *devInfo );
```

```
hipsolverStatus_t
hipsolverDnCsytrf( hipsolverDnHandle_t handle,
                   hipblasFillMode_t uplo,
                  int n,
                   hipFloatComplex *A,
                  int lda,
                  int *devIpiv,
                   hipFloatComplex *work,
                  int lwork,
                  int *devInfo );

hipsolverStatus_t
hipsolverDnZsytrf( hipsolverDnHandle_t handle,
                   hipblasFillMode_t uplo,
                  int n,
                   hipDoubleComplex *A,
                  int lda,
                  int *devIpiv,
                   hipDoubleComplex *work,
                  int lwork,
                  int *devInfo );
/*
This function computes the factorization of a symmetric indefinite matrix using the
Bunch-Kaufman diagonal pivoting.
*/
```

4.1.35 hipsolverDnIRSXgesv ()

```
hipsolverStatus_t
hipsolverDnIRSXgesv_bufferSize(
            hipsolverDnHandle_t                handle,
```

```
            hipsolverDnIRSParams_t          gesv_irs_params,
            hipsolver_int_t                 n,
            hipsolver_int_t                 nrhs,
            size_t                      *   lwork_bytes);
hipsolverStatus_t hipsolverDnIRSXgesv(
            hipsolverDnHandle_t             handle,
            hipsolverDnIRSParams_t          gesv_irs_params,
            hipsolverDnIRSInfos_t           gesv_irs_infos,
            int                             n,
            int                             nrhs,
            void                        *   dA,
            int                             ldda,
            void                        *   dB,
            int                             lddb,
            void                        *   dX,
            int                             lddx,
            void                        *   dWorkspace,
            size_t                          lwork_bytes,
            int                         *   dinfo);
/*
This function is designed to perform same functionality as hipsolverDn<T1><T2>gesv() functions, but wrapped in a more generic and expert interface that gives user more control to parametrize the function as well as it provides more information on output.  hipsolverDnIRSXgesv() allows additional control of the solver parameters such as setting:
-  the main precision (Inputs/Outputs precision) of the solver
-  the lowest precision to be used internally by the solver
-  the refinement solver type
-  the maximum allowed number of iterations in the refinement phase
```

- the tolerance of the refinement solver
- the fallback to main precision
- and more

through the configuration parameters structure gesv_irs_params and its helper functions.

*/

4.1.36 hipsolverDnIRSXgels ()

```
hipsolverStatus_t
hipsolverDnIRSXgels_bufferSize(
            hipsolverDnHandle_t              handle,
            hipsolverDnIRSParams_t           gels_irs_params,
            hipsolver_int_t                  m,
            hipsolver_int_t                  n,
            hipsolver_int_t                  nrhs,
           size_t                      *     lwork_bytes);
hipsolverStatus_t    hipsolverDnIRSXgels(
            hipsolverDnHandle_t              handle,
            hipsolverDnIRSParams_t           gels_irs_params,
            hipsolverDnIRSInfos_t            gels_irs_infos,
           int                              m,
           int                              n,
           int                              nrhs,
           void                        *     dA,
           int                              ldda,
           void                        *     dB,
           int                              lddb,
           void                        *     dX,
           int                              lddx,
           void                        *     dWorkspace,
```

```
        size_t                          lwork_bytes,
        int                         *   dinfo);
/*
This function is designed to perform same functionality as hipsolverDn<T1><T2>gels() functions, but wrapped in a more generic and expert interface that gives user more control to parametrize the function as well as it provides more information on output.  hipsolverDnIRSXgels() allows additional control of the solver parameters such as setting:
- the main precision (Inputs/Outputs precision) of the solver,
- the lowest precision to be used internally by the solver,
- the refinement solver type
- the maximum allowed number of iterations in the refinement phase
- the tolerance of the refinement solver
- the fallback to main precision
- and others
through the configuration parameters structure gels_irs_params and its helper functions.
*/
```

4.1.37 hipsolverDnXgesvdp()

```
hipsolverStatus_t
hipsolverDnXgesvdp_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverDnParams_t params,
      hipsolverEigMode_t jobz,
    int econ,
    int64_t m,
    int64_t n,
      hipDataType dataTypeA,
    const void *A,
    int64_t lda,
```

```
  hipDataType dataTypeS,
const void *S,
  hipDataType dataTypeU,
const void *U,
int64_t ldu,
  hipDataType dataTypeV,
const void *V,
int64_t ldv,
  hipDataType computeType,
size_t *workspaceInBytesOnDevice,
size_t *workspaceInBytesOnHost)
  hipsolverStatus_t
  hipsolverDnXgesvdp(
        hipsolverDnHandle_t handle,
        hipsolverDnParams_t params,
        hipsolverEigMode_t jobz,
      int econ,
      int64_t m,
      int64_t n,
        hipDataType dataTypeA,
      void *A,
      int64_t lda,
        hipDataType dataTypeS,
      void *S,
        hipDataType dataTypeU,
      void *U,
      int64_t ldu,
        hipDataType dataTypeV,
      void *V,
```

```
            int64_t ldv,
             hipDataType computeType,
            void *bufferOnDevice,
            size_t workspaceInBytesOnDevice,
            void *bufferOnHost,
            size_t workspaceInBytesOnHost,
            int *d_info,
            double *h_err_sigma)
/*
This function computes the singular value decomposition (SVD) of an m × n matrix
A and corresponding the left and/or right singular vectors. The SVD is written
                                A = U ∗ Σ ∗ V^H
where Σ is an m × n matrix which is zero except for its min(m,n) diagonal elements,
U is an m × m unitary matrix, and V is an n × n unitary matrix.
*/
```

4.1.38 hipsolverDnXgesvdr()

```
hipsolverStatus_t
hipsolverDnXgesvdr_bufferSize (
       hipsolverDnHandle_t handle,
       hipsolverDnParams_t params,
      signed char jobu,
      signed char jobv,
      int64_t m,
      int64_t n,
      int64_t k,
      int64_t p,
      int64_t niters,
       hipDataType dataTypeA,
      const void *A,
```

```
int64_t lda,
  hipDataType dataTypeSrand,
const void *Srand,
  hipDataType dataTypeUrand,
const void *Urand,
int64_t ldUrand,
  hipDataType dataTypeVrand,
const void *Vrand,
int64_t ldVrand,
  hipDataType computeType,
size_t *workspaceInBytesOnDevice,
size_t *workspaceInBytesOnHost )
  hipsolverStatus_t
  hipsolverDnXgesvdr(
        hipsolverDnHandle_t handle,
        hipsolverDnParams_t params,
       signed char jobu,
       signed char jobv,
       int64_t m,
       int64_t n,
       int64_t k,
       int64_t p,
       int64_t niters,
        hipDataType dataTypeA,
       void *A,
       int64_t lda,
        hipDataType dataTypeSrand,
       void *Srand,
        hipDataType    dataTypeUrand,
```

```
            void *Urand,
            int64_t ldUrand,
             hipDataType dataTypeVrand,
            void *Vrand,
            int64_t ldVrand,
             hipDataType computeType,
            void *bufferOnDevice,
            size_t workspaceInBytesOnDevice,
            void *bufferOnHost,
            size_t workspaceInBytesOnHost,
            int *d_info)
/*
This function computes the approximated rank-k singular value decomposition (k-SVD)
of an  m × n  matrix A and the corresponding left and/or right singular vectors. The k-
SVD is written as
                                  A_k ≈ U * Σ * V^H
where Σ is a  k × k  matrix which is zero except for its diagonal elements, U is an
m × k  orthonormal matrix, and V is an  k × n  orthonormal matrix.
*/
```

4.1.39 hipsolverDnXtrtri()

```
 hipsolverStatus_t
 hipsolverDnXtrtri_bufferSize(
        hipsolverDnHandle_t handle,
        hipblasFillMode_t uplo,
        hipblasDiagType_t diag,
       int64_t n,
        hipDataType dataTypeA,
       void *A,
       int64_t lda,
```

```
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost);
     hipsolverStatus_t
     hipsolverDnXtrtri(
           hipsolverDnHandle_t handle,
           hipblasFillMode_t uplo,
           hipblasDiagType_t diag,
         int64_t n,
           hipDataType dataTypeA,
         void *A,
         int64_t lda,
         void *bufferOnDevice,
         size_t workspaceInBytesOnDevice,
         void *bufferOnHost,
         size_t workspaceInBytesOnHost,
         int *info);
/*
The helper functions below can calculate the sizes needed for pre-allocated buffers. The following routine computes the inverse of a triangular matrix using the generic API interface. A is an  n × n  triangular matrix, only lower or upper part is meaningful. The input parameter uplo indicates which part of the matrix is used. The function will leave the other part untouched.
*/
```

4.1.40 hipsolverDnXsytrs()

```
 hipsolverStatus_t
 hipsolverDnXsytrs_bufferSize(
       hipsolverDnHandle_t handle,
       hipblasFillMode_t uplo,
     int64_t n,
```

```
        int64_t nrhs,
          hipDataType dataTypeA,
        const void *A,
        int64_t lda,
        const int64_t *devIpiv,
          hipDataType dataTypeB,
        void *B,
        int64_t ldb,
        size_t *workspaceInBytesOnDevice,
        size_t *workspaceInBytesOnHost);
          hipsolverStatus_t    hipsolverDnXsytrs(
                hipsolverDnHandle_t handle,
                hipblasFillMode_t uplo,
               int64_t n,
               int64_t nrhs,
                hipDataType dataTypeA,
               const void *A,
               int64_t lda,
               const int64_t *devIpiv,
                hipDataType dataTypeB,
               void *B,
               int64_t ldb,
               void *bufferOnDevice,
               size_t workspaceInBytesOnDevice,
               void *bufferOnHost,
               size_t workspaceInBytesOnHost,
               int *devInfo);
/*
The helper functions below can calculate the sizes needed for pre-allocated buffers. The
```

following routine solves a system of linear equations using the generic API interface. A contains the factorization from hipsolverDnXsytrf(), only lower or upper part is meaningful, the other part is not touched.
*/

4.1.41 hipsolverDnXgeev()

```
hipsolverStatus_t
hipsolverDnXgeev_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverDnParams_t params,
      hipsolverEigMode_t jobvl,
      hipsolverEigMode_t jobvr,
    int64_t n,
      hipDataType dataTypeA,
    const void *A,
    int64_t lda,
      hipDataType dataTypeW,
    const void *W,
      hipDataType dataTypeVL,
    const void *VL,
    int64_t ldvl,
      hipDataType dataTypeVR,
    const void *VR,
    int64_t ldvr,
      hipDataType computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost);
hipsolverStatus_t
hipsolverDnXgeev(
      hipsolverDnHandle_t handle,
```

```
    hipsolverDnParams_t params,
    hipsolverEigMode_t jobvl,
    hipsolverEigMode_t jobvr,
    int64_t n,
    hipDataType dataTypeA,
    void *A,
    int64_t lda,
    hipDataType dataTypeW,
    void *W,
    hipDataType dataTypeVL,
    void *VL,
    int64_t ldvl,
    hipDataType dataTypeVR,
    void *VR,
    int64_t ldvr,
    hipDataType computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost,
    int *info)
/*
```

The helper functions below can calculate the sizes needed for pre-allocated buffer. The following routine computes for an n-by-n real non-symmetric or complex non-Hermitian matrix A the eigenvalues and, optionally, the left and/or right eigenvectors. The right eigenvector v(j) of A satisfies

$$A * v(j) = w(j) * v(j)$$

where w(j) is its eigenvalue. The left eigenvalue u(j) of A satisfies

$$u(j)^H * A = w(j) * v(j)^H$$

where $u(j)^H$ denotes the conjugate-transpose of u(j).

*/

4.1.42 hipsolverDnXlarft()

```
hipsolverStatus_t    hipsolverDnXlarft_bufferSize(
      hipsolverDnHandle_t handle,
      hipsolverDnParams_t params,
      hipsolverDirectMode_t direct,
      hipsolverStorevMode_t storev,
    int64_t n,
    int64_t k,
      hipDataType dataTypeV,
    const void *V,
    int64_t ldv,
      hipDataType dataTypeTau,
    const void *tau,
      hipDataType dataTypeT,
    void *T,
    int64_t ldt,
      hipDataType computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost);
hipsolverStatus_t    hipsolverDnXlarft(
      hipsolverDnHandle_t handle,
      hipsolverDnParams_t params,
      hipsolverDirectMode_t direct,
      hipsolverStorevMode_t storev,
    int64_t n,
    int64_t k,
      hipDataType dataTypeV,
```

```
    const void *V,
    int64_t ldv,
     hipDataType dataTypeTau,
    const void *tau,
     hipDataType dataTypeT,
    void *T,
    int64_t ldt,
     hipDataType computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost)
/*
The helper functions below can calculate the sizes needed for pre-allocated buffer. The following routine forms the triangular factor T of a real block reflector H of order n, which is defined as a product of k elementary reflectors.
*/
```

4.2 hipSolverSp

4.2.1 hipsolverSpCreate()

```
hipsolverStatus_t hipsolverSpCreate(hipsolverSpHandle_t* handle);
/*


*/
```

4.2.2 hipsolverSpDestroy()

```
hipsolverStatus_t hipsolverSpDestroy(hipsolverSpHandle_t handle);
/*
```

```
*/
```

4.2.3 hipsolverSpSetStream()

```
hipsolverStatus_t hipsolverSpSetStream(hipsolverSpHandle_t handle,hipStream_t streamId);
/*
```

4.2.4 hipsolverSpGetStream()

```
hipsolverStatus_t
hipsolverSpGetStream(hipsolverSpHandle_t handle,
                                    hipStream_t *streamId)
/*
```

4.2.5 hipsolverSp <t>csrlsvlu()

```
hipsolverStatus_t
hipsolverSpScsrlsvlu[Host]( hipsolverSpHandle_t handle,
                        int n,
                        int nnzA,
                        const hipsparseMatDescr_t descrA,
                        const float *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const float *b,
                        float tol,
                        int reorder,
                        float *x,
                        int *singularity);

hipsolverStatus_t
hipsolverSpDcsrlsvlu[Host]( hipsolverSpHandle_t handle,
```

```
                        int n,
                        int nnzA,
                        const hipsparseMatDescr_t descrA,
                        const double *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const double *b,
                        double tol,
                        int reorder,
                        double *x,
                        int *singularity);

hipsolverStatus_t
hipsolverSpCcsrlsvlu[Host]( hipsolverSpHandle_t handle,
                        int n,
                        int nnzA,
                        const hipsparseMatDescr_t descrA,
                        const    hipFloatComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const    hipFloatComplex *b,
                        float tol,
                        int reorder,
                          hipFloatComplex *x,
                        int *singularity);

hipsolverStatus_t
hipsolverSpZcsrlsvlu[Host]( hipsolverSpHandle_t handle,
                        int n,
```

```
                        int nnzA,
                        const hipsparseMatDescr_t descrA,
                        const    hipDoubleComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const    hipDoubleComplex *b,
                        double tol,
                        int reorder,
                         hipDoubleComplex *x,
                        int *singularity);
/*
This function solves the linear system
                                    A * x = b
where A is an  n × n  sparse matrix that is defined in CSR storage format by the three
arrays csrValA, csrRowPtrA, and csrColIndA. b is the right-hand-side vector of size n,
and x is the solution vector of size n.
*/
```

4.2.6 hipsolverSp <t>csrlsvqr()

```
 hipsolverStatus_t
 hipsolverSpScsrlsvqr[Host]( hipsolverSpHandle_t handle,
                        int m,
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const float *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const float *b,
                        float tol,
                        int reorder,
```

```
                        float *x,
                        int *singularity);

hipsolverStatus_t
hipsolverSpDcsrlsvqr[Host]( hipsolverSpHandle_t handle,
                        int m,
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const double *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const double *b,
                        double tol,
                        int reorder,
                        double *x,
                        int *singularity);

hipsolverStatus_t
hipsolverSpCcsrlsvqr[Host]( hipsolverSpHandle_t handle,
                        int m,
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const    hipFloatComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const    hipFloatComplex *b,
                        float tol,
                        int reorder,
                         hipFloatComplex *x,
```

```
                        int *singularity);


 hipsolverStatus_t
 hipsolverSpZcsrlsvqr[Host]( hipsolverSpHandle_t handle,
                        int m,
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const    hipDoubleComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const    hipDoubleComplex *b,
                        double tol,
                        int reorder,
                          hipDoubleComplex *x,
                        int *singularity);
/*
This function solves the linear system
                                    A * x = b
A is an  m × m  sparse matrix that is defined in CSR storage format by the three arrays
csrValA, csrRowPtrA, and csrColIndA. b is the right-hand-side vector of size m, and x
is the solution vector of size m.
*/
```

4.2.7 hipsolverSp <t>csrlsvchol()

```
 hipsolverStatus_t
 hipsolverSpScsrlsvchol[Host]( hipsolverSpHandle_t handle,
                              int m,
                              int nnz,
                              const hipsparseMatDescr_t descrA,
                              const float *csrVal,
```

```
                            const int *csrRowPtr,
                            const int *csrColInd,
                            const float *b,
                            float tol,
                            int reorder,
                            float *x,
                            int *singularity);

hipsolverStatus_t
hipsolverSpDcsrlsvchol[Host]( hipsolverSpHandle_t handle,
                            int m,
                            int nnz,
                            const hipsparseMatDescr_t descrA,
                            const double *csrVal,
                            const int *csrRowPtr,
                            const int *csrColInd,
                            const double *b,
                            double tol,
                            int reorder,
                            double *x,
                            int *singularity);

hipsolverStatus_t
hipsolverSpCcsrlsvchol[Host]( hipsolverSpHandle_t handle,
                            int m,
                            int nnz,
                            const hipsparseMatDescr_t descrA,
                            const    hipFloatComplex *csrVal,
                            const int *csrRowPtr,
```

```
                                const int *csrColInd,
                                const    hipFloatComplex *b,
                                float tol,
                                int reorder,
                                 hipFloatComplex *x,
                                int *singularity);


 hipsolverStatus_t
 hipsolverSpZcsrlsvchol[Host]( hipsolverSpHandle_t handle,
                                int m,
                                int nnz,
                                const hipsparseMatDescr_t descrA,
                                const    hipDoubleComplex *csrVal,
                                const int *csrRowPtr,
                                const int *csrColInd,
                                const    hipDoubleComplex *b,
                                double tol,
                                int reorder,
                                 hipDoubleComplex *x,
                                int *singularity);
/*
 This function solves the linear system
                                        A * x = b
A is an m × m symmetric positive definite sparse matrix that is defined in CSR storage format by the three arrays csrValA, csrRowPtrA, and csrColIndA. b is the right-hand-side vector of size m, and x is the solution vector of size m.
*/
```

4.2.8 hipsolverSp <t>csrlsqvqr()

```
 hipsolverStatus_t
```

```
hipsolverSpScsrlsqvqr[Host]( hipsolverSpHandle_t handle,
                             int m,
                             int n,
                             int nnz,
                             const hipsparseMatDescr_t descrA,
                             const float *csrValA,
                             const int *csrRowPtrA,
                             const int *csrColIndA,
                             const float *b,
                             float tol,
                             int *rankA,
                             float *x,
                             int *p,
                             float *min_norm);

hipsolverStatus_t
hipsolverSpDcsrlsqvqr[Host]( hipsolverSpHandle_t handle,
                             int m,
                             int n,
                             int nnz,
                             const hipsparseMatDescr_t descrA,
                             const double *csrValA,
                             const int *csrRowPtrA,
                             const int *csrColIndA,
                             const double *b,
                             double tol,
                             int *rankA,
                             double *x,
                             int *p,
```

```
                        double *min_norm);
hipsolverStatus_t
hipsolverSpCcsrlsqvqr[Host]( hipsolverSpHandle_t handle,
                        int m,
                        int n,
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const    hipFloatComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const    hipFloatComplex *b,
                        float tol,
                        int *rankA,
                          hipFloatComplex *x,
                        int *p,
                        float *min_norm);

hipsolverStatus_t
hipsolverSpZcsrlsqvqr[Host]( hipsolverSpHandle_t handle,
                        int m,
                        int n,
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const    hipDoubleComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        const    hipDoubleComplex *b,
                        double tol,
                        int *rankA,
```

```
                          hipDoubleComplex *x,
                          int *p,
                          double *min_norm);
/*
This function solves the following least-square problem:
                                x = argmin||A * z − b||
A is an m × n sparse matrix that is defined in CSR storage format by the three arrays
csrValA, csrRowPtrA, and csrColIndA. b is the right-hand-side vector of size m, and x
is the least-square solution vector of size n.
*/
```

4.2.9 hipsolverSp <t>csreigvsi()

```
hipsolverStatus_t
hipsolverSpScsreigvsi[Host]( hipsolverSpHandle_t handle,
                          int m,
                          int nnz,
                          const hipsparseMatDescr_t descrA,
                          const float *csrValA,
                          const int *csrRowPtrA,
                          const int *csrColIndA,
                          float mu0,
                          const float *x0,
                          int maxite,
                          float tol,
                          float *mu,
                          float *x);

hipsolverStatus_t
hipsolverSpDcsreigvsi[Host]( hipsolverSpHandle_t handle,
                          int m,
```

```
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const double *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        double mu0,
                        const double *x0,
                        int maxite,
                        double tol,
                        double *mu,
                        double *x);
hipsolverStatus_t
hipsolverSpCcsreigvsi[Host]( hipsolverSpHandle_t handle,
                        int m,
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const    hipFloatComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                         hipFloatComplex mu0,
                        const    hipFloatComplex *x0,
                        int maxite,
                        float tol,
                         hipFloatComplex *mu,
                         hipFloatComplex *x);

hipsolverStatus_t
hipsolverSpZcsreigvsi( hipsolverSpHandle_t handle,
                        int m,
```

```
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const    hipDoubleComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                         hipDoubleComplex mu0,
                        const    hipDoubleComplex *x0,
                        int maxite,
                        double tol,
                         hipDoubleComplex *mu,
                         hipDoubleComplex *x);
/*
This function solves the simple eigenvalue problem A * x = λ * x by shift-inverse
method. A is an m × m sparse matrix that is defined in CSR storage format by the
three arrays csrValA, csrRowPtrA, and csrColIndA. The output parameter x is the
approximated eigenvector of size m.
*/
```

4.2.10 hipsolverSp <t>csreigs()

```
 hipsolverStatus_t
solverspScsreigs[Host]( hipsolverSpHandle_t handle,
                        int m,
                        int nnz,
                        const hipsparseMatDescr_t descrA,
                        const float *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                         hipFloatComplex left_bottom_corner,
                         hipFloatComplex right_upper_corner,
                        int *num_eigs);
```

```
hipsolverStatus_t
hipsolverSpDcsreigs[Host]( hipsolverSpHandle_t handle,
                           int m,
                           int nnz,
                           const hipsparseMatDescr_t descrA,
                           const double *csrValA,
                           const int *csrRowPtrA,
                           const int *csrColIndA,
                            hipDoubleComplex left_bottom_corner,
                            hipDoubleComplex right_upper_corner,
                           int *num_eigs);

hipsolverStatus_t
hipsolverSpCcsreigs[Host]( hipsolverSpHandle_t handle,
                           int m,
                           int nnz,
                           const hipsparseMatDescr_t descrA,
                           const    hipFloatComplex *csrValA,
                           const int *csrRowPtrA,
                           const int *csrColIndA,
                            hipFloatComplex left_bottom_corner,
                            hipFloatComplex right_upper_corner,
                           int *num_eigs);

hipsolverStatus_t
hipsolverSpZcsreigs[Host]( hipsolverSpHandle_t handle,
                           int m,
                           int nnz,
```

```
                        const hipsparseMatDescr_t descrA,
                        const    hipDoubleComplex *csrValA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                          hipDoubleComplex left_bottom_corner,
                          hipDoubleComplex right_upper_corner,
                        int *num_eigs);
/*
This function computes number of algebraic eigenvalues in a given box B by contour integral
```

$$\text{number of algebraic eigenvalues in box B} = \frac{1}{2 * \pi * \sqrt{-1}} \oint_C \frac{P'(z)}{P(z)} dz$$

```
where closed line C is boundary of the box B which is a rectangle specified by two points, one is left bottom corner (input parameter left_bottom_corner) and the other is right upper corner (input parameter right_upper_corner). P(z)=det(A - z*I) is the characteristic polynomial of A.
A is an m × m sparse matrix that is defined in CSR storage format by the three arrays csrValA, csrRowPtrA, and csrColIndA.
*/
```

4.2.11 hipsolverSpXcsrsymrcm()

```
 hipsolverStatus_t
 hipsolverSpXcsrsymrcmHost( hipsolverSpHandle_t handle,
                        int n,
                        int nnzA,
                        const hipsparseMatDescr_t descrA,
                        const int *csrRowPtrA,
                        const int *csrColIndA,
                        int *p);
/*
```

This function implements Symmetric Reverse Cuthill-McKee permutation. It returns a permutation vector p such that A(p,p) would concentrate nonzeros to diagonal.

The output parameter p is an integer array of n elements. It represents a permutation array and it indexed using the base-0 convention. The permutation array p corresponds to a permutation matrix P, and satisfies the following relation:

$$A(p,p) = P * A * P^T$$

A is an $n \times n$ sparse matrix that is defined in CSR storage format by the three arrays csrValA, csrRowPtrA, and csrColIndA.

*/

4.2.12 hipsolverSpXcsrsymmdq()

```
hipsolverStatus_t
hipsolverSpXcsrsymmdqHost( hipsolverSpHandle_t handle,
                           int n,
                           int nnzA,
                           const hipsparseMatDescr_t descrA,
                           const int *csrRowPtrA,
                           const int *csrColIndA,
                           int *p);
```

/*

This function implements Symmetric Minimum Degree Algorithm based on Quotient Graph. It returns a permutation vector p such that A(p,p) would have less zero fill-in during Cholesky factorization.

The output parameter p is an integer array of n elements. It represents a permutation array and it indexed using the base-0 index. The permutation array p corresponds to a permutation matrix P, and satisfies the following relation:

$$A(p,p) = P * A * P^T$$

A is an $n \times n$ sparse matrix that is defined in CSR storage format by the three arrays csrValA, csrRowPtrA, and csrColIndA.

*/

4.2.13 hipsolverSpXcsrsymamd()

```
hipsolverStatus_t
hipsolverSpXcsrsymamdHost( hipsolverSpHandle_t handle,
                    int n,
                    int nnzA,
                    const hipsparseMatDescr_t descrA,
                    const int *csrRowPtrA,
                    const int *csrColIndA,
                    int *p);
/*
This function implements Symmetric Approximate Minimum Degree Algorithm based on Quotient Graph. It returns a permutation vector p such that A(p,p) would have less zero fill-in during Cholesky factorization.
The output parameter p is an integer array of n elements. It represents a permutation array and it indexed using the base-0 index. The permutation array p corresponds to a permutation matrix P, and satisfies the following relation:
                                    A(p,p) = P * A * P^T
A is an n × n sparse matrix that is defined in CSR storage format by the three arrays csrValA, csrRowPtrA, and csrColIndA.
*/
```

4.2.14 hipsolverSpXcsrmetisnd()

```
hipsolverStatus_t
hipsolverSpXcsrmetisndHost(
      hipsolverSpHandle_t handle,
    int n,
    int nnzA,
    const hipsparseMatDescr_t descrA,
    const int *csrRowPtrA,
```

```
    const int *csrColIndA,
    const int64_t *options,
    int *p);
/*
This function is a wrapper of METIS_NodeND. It returns a permutation vector p such that A(p,p) would have less zero fill-in during nested dissection.
The output parameter p is an integer array of n elements. It represents a permutation array and it indexed using the base-0 index. The permutation array p corresponds to a permutation matrix P, and satisfies the following relation:
                                A(p,p) = P * A * P^T
A is an n × n sparse matrix that is defined in CSR storage format by the three arrays csrValA, csrRowPtrA, and csrColIndA.
*/
```

4.2.15 hipsolverSpXcsrzfd()

```
hipsolverStatus_t
hipsolverSpScsrzfdHost(
      hipsolverSpHandle_t handle,
    int n,
    int nnzA,
    const hipsparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *P,
    int *numnz)

hipsolverStatus_t
hipsolverSpDcsrzfdHost(
      hipsolverSpHandle_t handle,
```

```
    int n,
    int nnzA,
    const hipsparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *P,
    int *numnz)

hipsolverStatus_t
hipsolverSpCcsrzfdHost(
     hipsolverSpHandle_t handle,
    int n,
    int nnzA,
    const hipsparseMatDescr_t descrA,
    const    hipFloatComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *P,
    int *numnz)

hipsolverStatus_t
hipsolverSpZcsrzfdHost(
     hipsolverSpHandle_t handle,
    int n,
    int nnzA,
    const hipsparseMatDescr_t descrA,
    const    hipDoubleComplex *csrValA,
    const int *csrRowPtrA,
```

```
        const int *csrColIndA,
        int *P,
        int *numnz)
/*
This function implements MC21, zero-free diagonal algorithm. It returns a permutation
vector p such that A(p,:) has no zero diagonal.
The output parameter p is an integer array of n elements. It represents a permutation
array with base-0 index. The permutation array p corresponds to a permutation matrix
P, and satisfies the following relation:
                                    A(p,:) = P * A
The output parameter numnz describes number of nonzero diagonal in permutated
matrix A(p,:). If numnz is less than n, matrix A has structural singularity.
*/
```

4.2.16 hipsolverSpXcsrperm()

```
hipsolverStatus_t
hipsolverSpXcsrperm_bufferSizeHost( hipsolverSpHandle_t handle,
                                    int m,
                                    int n,
                                    int nnzA,
                                    const hipsparseMatDescr_t descrA,
                                    int *csrRowPtrA,
                                    int *csrColIndA,
                                    const int *p,
                                    const int *q,
                                    size_t *bufferSizeInBytes);

hipsolverStatus_t
hipsolverSpXcsrpermHost( hipsolverSpHandle_t handle,
                         int m,
```

```
                    int n,
                    int nnzA,
                    const hipsparseMatDescr_t descrA,
                    int *csrRowPtrA,
                    int *csrColIndA,
                    const int *p,
                    const int *q,
                    int *map,
                    void *pBuffer);
/*
Given a left permutation vector p which corresponds to permutation matrix P and a right permutation vector q which corresponds to permutation matrix Q, this function computes permutation of matrix A by
                                B = P * A * Q^T
A is an m × n sparse matrix that is defined in CSR storage format by the three arrays csrValA, csrRowPtrA and csrColIndA.
*/
```

4.2.17 hipsolverSpXcsrqrBatched()

```
hipsolverStatus_t
hipsolverSpCreateCsrqrInfo(csrqrInfo_t *info);

hipsolverStatus_t
hipsolverSpDestroyCsrqrInfo(csrqrInfo_t info);
hipsolverStatus_t
hipsolverSpXcsrqrAnalysisBatched( hipsolverSpHandle_t handle,
                                  int m,
                                  int n,
                                  int nnzA,
                                  const hipsparseMatDescr_t descrA,
```

```
                                    const int *csrRowPtrA,
                                    const int *csrColIndA,
                                    csrqrInfo_t info);

hipsolverStatus_t
hipsolverSpScsrqrBufferInfoBatched( hipsolverSpHandle_t handle,
                                    int m,
                                    int n,
                                    int nnzA,
                                    const hipsparseMatDescr_t descrA,
                                    const float *csrValA,
                                    const int *csrRowPtrA,
                                    const int *csrColIndA,
                                    int batchSize,
                                    csrqrInfo_t info,
                                    size_t *internalDataInBytes,
                                    size_t *workspaceInBytes);

hipsolverStatus_t
hipsolverSpDcsrqrBufferInfoBatched( hipsolverSpHandle_t handle,
                                    int m,
                                    int n,
                                    int nnzA,
                                    const hipsparseMatDescr_t descrA,
                                    const double *csrValA,
                                    const int *csrRowPtrA,
                                    const int *csrColIndA,
                                    int batchSize,
                                    csrqrInfo_t info,
```

```
                                    size_t *internalDataInBytes,
                                    size_t *workspaceInBytes);
hipsolverStatus_t
hipsolverSpCcsrqrBufferInfoBatched( hipsolverSpHandle_t handle,
                                    int m,
                                    int n,
                                    int nnzA,
                                    const hipsparseMatDescr_t descrA,
                                    const    hipFloatComplex *csrValA,
                                    const int *csrRowPtrA,
                                    const int *csrColIndA,
                                    int batchSize,
                                    csrqrInfo_t info,
                                    size_t *internalDataInBytes,
                                    size_t *workspaceInBytes);

hipsolverStatus_t
hipsolverSpZcsrqrBufferInfoBatched( hipsolverSpHandle_t handle,
                                    int m,
                                    int n,
                                    int nnzA,
                                    const hipsparseMatDescr_t descrA,
                                    const    hipDoubleComplex *csrValA,
                                    const int *csrRowPtrA,
                                    const int *csrColIndA,
                                    int batchSize,
                                    csrqrInfo_t info,
                                    size_t *internalDataInBytes,
                                    size_t *workspaceInBytes);
```

```
hipsolverStatus_t
hipsolverSpScsrqrsvBatched( hipsolverSpHandle_t handle,
                            int m,
                            int n,
                            int nnzA,
                            const hipsparseMatDescr_t descrA,
                            const float *csrValA,
                            const int *csrRowPtrA,
                            const int *csrColIndA,
                            const float *b,
                            float *x,
                            int batchSize,
                            csrqrInfo_t info,
                            void *pBuffer);

hipsolverStatus_t
hipsolverSpDcsrqrsvBatched( hipsolverSpHandle_t handle,
                            int m,
                            int n,
                            int nnz,
                            const hipsparseMatDescr_t descrA,
                            const double *csrValA,
                            const int *csrRowPtrA,
                            const int *csrColIndA,
                            const double *b,
                            double *x,
                            int batchSize,
                            csrqrInfo_t info,
                            void *pBuffer);
```

```
hipsolverStatus_t
hipsolverSpCcsrqrsvBatched( hipsolverSpHandle_t handle,
                            int m,
                            int n,
                            int nnzA,
                            const hipsparseMatDescr_t descrA,
                            const    hipFloatComplex *csrValA,
                            const int *csrRowPtrA,
                            const int *csrColIndA,
                            const    hipFloatComplex *b,
                             hipFloatComplex *x,
                            int batchSize,
                            csrqrInfo_t info,
                            void *pBuffer);
hipsolverStatus_t
hipsolverSpZcsrqrsvBatched( hipsolverSpHandle_t handle,
                            int m,
                            int n,
                            int nnzA,
                            const hipsparseMatDescr_t descrA,
                            const    hipDoubleComplex *csrValA,
                            const int *csrRowPtrA,
                            const int *csrColIndA,
                            const    hipDoubleComplex *b,
                             hipDoubleComplex *x,
                            int batchSize,
                            csrqrInfo_t info,
                            void *pBuffer);
/*
```

```
The batched sparse QR factorization is used to solve either a set of least-squares problems or a set of linear systems where each A_j is an m × n sparse matrix that is defined in CSR storage format by the four arrays csrValA, csrRowPtrA and csrColIndA.
*/
```

4.2.18 hipsolverspXcsrissymHost ()

```
hipsolverStatus_t
hipsolverspXcsrissymHost(hipsolverSpHandle_t handle,
                                                    int m,
                                                    int nnzA,
                                                    const hipsparseMatDescr_t descrA,
                                                    const int *csrRowPtrA,
                                                    const int *csrEndPtrA,
                                                    const int *csrColIndA,
                                                    int *issym)
/*
This function checks if A has symmetric pattern or not. The output parameter issym reports 1 if A is symmetric; otherwise, it reports 0.
*/
```

4.3 hipSolverRf

4.3.1 hipsolverRfAccessBundledFactorsDevice()

```
hipsolverStatus_t
hipsolverRfAccessBundledFactorsDevice(/* Input */
                                        hipsolverRfHandle_t handle,
                                      /* Output (in the host memory) */
                                      int* nnzM,
                                      /* Output (in the device memory) */
                                      int** Mp,
                                      int** Mi,
```

double** Mx);

/*

This routine allows direct access to the lower L and upper U triangular factors stored in the hipsolverRF library handle. The factors are compressed into a single matrix M=(L-I)+U, where the unitary diagonal of L is not stored. It is assumed that a prior call to the hipsolverRfRefactor() was done in order to generate these triangular factors.

*/

4.3.2 hipsolverRfAnalyze()

```
hipsolverStatus_t hipsolverRfAnalyze( hipsolverRfHandle_t handle);
/*
This routine performs the appropriate analysis of parallelism available in the LU re-factorization depending upon the algorithm chosen by the user.
                                    A = L * U
It is assumed that a prior call to the hipsolverRfSetup[Host|Device]() was done in order to create internal data structures needed for the analysis.
This routine needs to be called only once for a single linear system
                                    A_i x_i = f_i
*/
```

4.3.3 hipsolverRfSetupDevice()

```
hipsolverStatus_t
hipsolverRfSetupDevice(/* Input (in the device memory) */
                       int n,
                       int nnzA,
                       int* csrRowPtrA,
                       int* csrColIndA,
                       double* csrValA,
                       int nnzL,
                       int* csrRowPtrL,
```

```
                        int* csrColIndL,
                        double* csrValL,
                        int nnzU,
                        int* csrRowPtrU,
                        int* csrColIndU,
                        double* csrValU,
                        int* P,
                        int* Q,
                        /* Output */
                          hipsolverRfHandle_t handle);
/*
This routine assembles the internal data structures of the  hipsolverRF library. It is
often the first routine to be called after the call to the  hipsolverRfCreate() routine.
This routine accepts as input (on the device) the original matrix A, the lower (L) and
upper (U) triangular factors, as well as the left (P) and the right (Q) permutations
resulting from the full LU factorization of the first (i=1) linear system
                                    A_i x_i = f_i
*/
```

4.3.4 hipsolverRfSetupHost()

```
hipsolverStatus_t
hipsolverRfSetupHost(/* Input (in the host memory) */
                        int n,
                        int nnzA,
                        int* h_csrRowPtrA,
                        int* h_csrColIndA,
                        double* h_csrValA,
                        int nnzL,
                        int* h_csrRowPtrL,
                        int* h_csrColIndL,
```

```
                        double* h_csrValL,
                        int nnzU,
                        int* h_csrRowPtrU,
                        int* h_csrColIndU,
                        double* h_csrValU,
                        int* h_P,
                        int* h_Q,
                        /* Output */
                         hipsolverRfHandle_t handle);
/*
This routine assembles the internal data structures of the  hipsolverRF library. It is often the first routine to be called after the call to the  hipsolverRfCreate() routine.
This routine accepts as input (on the host) the original matrix A, the lower (L) and upper (U) triangular factors, as well as the left (P) and the right (Q) permutations resulting from the full LU factorization of the first (i=1) linear system
                                    A_i x_i = f_i
*/
```

4.3.5 hipsolverRfCreate()

```
 hipsolverStatus_t  hipsolverRfCreate( hipsolverRfHandle_t *handle);
/*
This routine initializes the  hipsolverRF library. It allocates required resources and must be called prior to any other  hipsolverRF library routine.
*/
```

4.3.6 hipsolverRfDestroy()

```
 hipsolverStatus_t  hipsolverRfDestroy( hipsolverRfHandle_t handle);
/*
This routine shuts down the  hipsolverRF library. It releases acquired resources and must be called after all the  hipsolverRF library routines.
```

```
*/
```

4.3.7 hipsolverRfExtractBundledFactorsHost()

```
hipsolverStatus_t
hipsolverRfExtractBundledFactorsHost(/* Input */
                                     hipsolverRfHandle_t handle,
                                     /* Output (in the host memory) */
                                     int* h_nnzM,
                                     int** h_Mp,
                                     int** h_Mi,
                                     double** h_Mx);
/*
This routine extracts lower (L) and upper (U) triangular factors from the hipsolverRF library handle into the host memory. The factors are compressed into a single matrix M=(L-I)+U, where the unitary diagonal of (L) is not stored. It is assumed that a prior call to the hipsolverRfRefactor() was done in order to generate these triangular factors.
*/
```

4.3.8 hipsolverRfExtractSplitFactorsHost()

```
hipsolverStatus_t
hipsolverRfExtractSplitFactorsHost(/* Input */
                                   hipsolverRfHandle_t handle,
                                   /* Output (in the host memory) */
                                   int* h_nnzL,
                                   int** h_Lp,
                                   int** h_Li,
                                   double** h_Lx,
                                   int* h_nnzU,
                                   int** h_Up,
                                   int** h_Ui,
```

```
                                        double** h_Ux);
/*
This routine extracts lower (L) and upper (U) triangular factors from the    hipsolverRF
library handle into the host memory. It is assumed that a prior call to the
hipsolverRfRefactor() was done in order to generate these triangular factors.
*/
```

4.3.9 hipsolverRfGet_Algs

```
 hipsolverStatus_t
 hipsolverRfGetAlgs( hipsolverRfHandle_t handle,
                     hipsolverRfFactorization_t* fact_alg,
                     hipsolverRfTriangularSolve_t* solve_alg);
/*
This routine gets the algorithm used for the refactorization in    hipsolverRfRefactor()
and the triangular solve in    hipsolverRfSolve().
*/
```

4.3.10 hipsolverRfGetAlgs

```
 hipsolverStatus_t
 hipsolverRfGetAlgs(hipsolverRfHandle_t                handle,
                    hipsolverRfFactorization_t*      fact_alg,
                    hipsolverRfTriangularSolve_t* solve_alg)
/*
This routine gets the algorithm used for the refactorization in    hipsolverRfRefactor()
and the triangular solve in    hipsolverRfSolve().
*/
```

4.3.11 hipsolverRfGetMatrixFormat

```
 hipsolverStatus_t
 hipsolverRfGetMatrixFormat( hipsolverRfHandle_t handle,
                             hipsolverRfMatrixFormat_t *format,
```

```
                                    hipsolverRfUnitDiagonal_t *diag);
/*
This routine gets the matrix format used in the hipsolverRfSetupDevice(),
hipsolverRfSetupHost(), hipsolverRfResetValues(),
hipsolverRfExtractBundledFactorsHost() and hipsolverRfExtractSplitFactorsHost()
routines.
*/
```

4.3.12 hipsolverRfGetNumericBoostReport

```
hipsolverStatus_t
hipsolverRfGetNumericBoostReport( hipsolverRfHandle_t handle,
                                  hipsolverRfNumericBoostReport_t *report);
/*
This routine gets the report whether numeric boosting was used in the
hipsolverRfRefactor() and hipsolverRfSolve() routines.
*/
```

4.3.13 hipsolverRfGetNumericProperties

```
hipsolverStatus_t
hipsolverRfGetNumericProperties( hipsolverRfHandle_t handle,
                                 double *zero,
                                 double *boost);
/*
This routine gets the numeric values used for checking for ‘’zero” pivot and for
boosting it in the hipsolverRfRefactor() and hipsolverRfSolve() routines. The
numeric boosting will be used only if boost > 0.0.
*/
```

4.3.14 hipsolverRfGetResetValuesFastMode

```
hipsolverStatus_t
hipsolverRfGetResetValuesFastMode( hipsolverRfHandle_t handle,
```

```
                                        hipsolverRfResetValuesFastMode_t
*fastMode);
/*
This routine gets the mode used in the    hipsolverRfResetValues routine.
*/
```

4.3.15 hipsolverRfRefactor

```
 hipsolverStatus_t    hipsolverRfRefactor( hipsolverRfHandle_t handle);
/*
This routine performs the LU re-factorization:
                                    A = L * U
exploring the available parallelism on the DCU. It is assumed that a prior call to the
hipsolverRfAnalyze() was done in order to find the available parallelism.
                                    A_i x_i = f_i
This routine may be called multiple times, once for each of the linear systems:
*/
```

4.3.16 hipsolverRfResetValues

```
 hipsolverStatus_t
 hipsolverRfResetValues(/* Input (in the device memory) */
                        int n,
                        int nnzA,
                        int* csrRowPtrA,
                        int* csrColIndA,
                        double* csrValA,
                        int* P,
                        int* Q,
                        /* Output */
                         hipsolverRfHandle_t handle);
/*
```

This routine updates internal data structures with the values of the new coefficient matrix. It is assumed that the arrays csrRowPtrA, csrColIndA, P and Q have not changed since the last call to the hipsolverRfSetup[Host|Device] routine. This assumption reflects the fact that the sparsity pattern of coefficient matrices as well as reordering to minimize fill-in and pivoting remain the same in the set of linear systems:

$$A_i x_i = f_i$$

This routine may be called multiple times, once for each of the linear systems:

$$A_i x_i = f_i$$

*/

4.3.17 hipsolverRfSetAlgs

```
hipsolverStatus_t
hipsolverRfSetAlgs( hipsolverRfHandle_t handle,
                    hipsolverRfFactorization_t fact_alg,
                    hipsolverRfTriangularSolve_t alg);
/*
This routine sets the algorithm used for the refactorization in hipsolverRfRefactor()
and the triangular solve in hipsolverRfSolve(). It may be called once prior to
hipsolverRfAnalyze() routine.
*/
```

4.3.18 hipsolverRfSetMatrixFormat

```
hipsolverStatus_t
hipsolverRfSetMatrixFormat( hipsolverRfHandle_t handle,
                            hipsolverRfMatrixFormat_t format,
                            hipsolverRfUnitDiagonal_t diag);
/*
This routine sets the matrix format used in the hipsolverRfSetupDevice(),
hipsolverRfSetupHost(), hipsolverRfResetValues(),
hipsolverRfExtractBundledFactorsHost() and hipsolverRfExtractSplitFactorsHost()
```

```
routines. It may be called once prior to hipsolverRfSetupDevice() and
hipsolverRfSetupHost() routines.
*/
```

4.3.19 hipsolverRfSetNumericProperties

```
hipsolverStatus_t
hipsolverRfSetNumericProperties( hipsolverRfHandle_t handle,
                                 double zero,
                                 double boost);
/*
This routine sets the numeric values used for checking for ‘’zero’’ pivot and for boosting
it in the hipsolverRfRefactor() and hipsolverRfSolve() routines. It may be called
multiple times prior to hipsolverRfRefactor() and hipsolverRfSolve() routines. The
numeric boosting will be used only if boost > 0.0.
*/
```

4.3.20 hipsolverRfSetResetValuesFastMode

```
hipsolverStatus_t
hipsolverRfSetResetValuesFastMode( hipsolverRfHandle_t handle,
                                   hipsolverRfResetValuesFastMode_t
fastMode); /*
This routine sets the mode used in the hipsolverRfResetValues routine. The fast mode
requires extra memory and is recommended only if very fast calls to
hipsolverRfResetValues() are needed. It may be called once prior to
hipsolverRfAnalyze() routine.
*/
```

4.3.21 hipsolverRfSolve

```
hipsolverStatus_t
hipsolverRfSolve(/* Input (in the device memory) */
                 hipsolverRfHandle_t handle,
```

```
                int *P,
                int *Q,
                int nrhs,
                double *Temp,
                int ldt,
                /* Input/Output (in the device memory) */
                double *XF,
                /* Input */
                int ldxf);
/*
This routine performs the forward and backward solve with the lower L ∈ R^nxn and
upper U ∈ R^nxn triangular factors resulting from the LU re-factorization:
                                A = L * U
which is assumed to have been computed by a prior call to the hipsolverRfRefactor()
routine.
*/
```

4.3.22 hipsolverRfBatchSetupHost

```
hipsolverStatus_t
hipsolverRfBatchSetupHost(/* Input (in the host memory) */
                                int batchSize,
                                int n,
                                int nnzA,
                                int* h_csrRowPtrA,
                                int* h_csrColIndA,
                                double *h_csrValA_array[],
                                int nnzL,
                                int* h_csrRowPtrL,
                                int* h_csrColIndL,
                                double *h_csrValL,
```

```
                                int nnzU,
                                int* h_csrRowPtrU,
                                int* h_csrColIndU,
                                double *h_csrValU,
                                int* h_P,
                                int* h_Q,
                                /* Output */
                                 hipsolverRfHandle_t handle);
/*
This routine assembles the internal data structures of the  hipsolverRF library for batched operation. It is called after the call to the  hipsolverRfCreate() routine, and before any other batched routines.
The batched operation assumes that the user has the following linear systems:
                                A_j x_j = b_j, j = 1,2, ..., batchSize
where each matrix in the set: {A_j} has the same sparsity pattern, and quite similar such that factorization can be done by the same permutation P and Q.
*/
```

4.3.23 hipsolverRfBatchAnalyze

```
 hipsolverStatus_t  hipsolverRfBatchAnalyze( hipsolverRfHandle_t handle);
/*
This routine performs the appropriate analysis of parallelism available in the batched LU re-factorization.
It is assumed that a prior call to the  hipsolverRfBatchSetup[Host]() was done in order to create internal data structures needed for the analysis.
The following routine needs to be called only once for a single linear system:
                                A_j x_j = b_j, j = 1,2, ..., batchSize
*/
```

4.3.24 hipsolverRfBatchRefactor

```
hipsolverStatus_t    hipsolverRfBatchRefactor( hipsolverRfHandle_t handle);
/*
This routine performs the LU re-factorization:
```

$$M_j = P * A_j * Q^T = L_j * U_j$$

exploring the available parallelism on the DCU. It is assumed that a prior call to the hipsolverRfBatchAnalyze() was done in order to find the available parallelism.

```
*/
```

4.3.25 hipsolverRfBatchSolve

```
hipsolverStatus_t
hipsolverRfBatchSolve(/* Input (in the device memory) */
                      hipsolverRfHandle_t handle,
                      int *P,
                      int *Q,
                      int nrhs,
                      double *Temp,
                      int ldt,
                      /* Input/Output (in the device memory) */
                      double *XF_array[],
                      /* Input */
                      int ldxf);
/*
To solve
```

$A_j * x_j = b_j$, first we reform the equation by $M_j * Q * x_j = P * b_j$ where . Then do refactorization $M_j = P * A_j * Q^T$ by hipsolverRfBatch_Refactor(). Further hipsolverRfBatch_Solve() takes over the remaining steps, including:

$$z_j = P * b_j$$

$$M_j * y_j = z_j$$

$$x_j = Q^T * y_j$$

```
The input parameter XF_array is an array of pointers on device memory. XF_array(j) points to matrix x_j which is also on device memory.
*/
```

4.3.26 hipsolverRfBatchZeroPivot

```
hipsolverStatus_t
hipsolverRfBatchZeroPivot(/* Input */
                          hipsolverRfHandle_t handle
                         /* Output (in the host memory) */
                         int *position);
/*
Although A_j is close to each other, it does not mean M_j = P * A_j * Q^T = L_j * U_j exists for every j. The user can query which matrix failed LU refactorization by checking corresponding value in position array. The input parameter position is an integer array of size batchSize.
*/
```

4.4 hipSolverMG

4.4.1 hipsolverMgCreate()

```
hipsolverStatus_t  hipsolverMgCreate( hipsolverMgHandle_t *handle)
/*
This function initializes the hipsolverMG library and creates a handle on the hipsolverMG context. It must be called before any other hipsolverMG API function is invoked. It allocates hardware resources necessary for accessing the DCU.
*/
```

4.4.2 hipsolverMgDestroy()

```
hipsolverStatus_t  hipsolverMgDestroy(  hipsolverMgHandle_t handle)
/*
```

```
This function releases CPU-side resources used by the    hipsolverMG library.
*/
```

4.4.3 hipsolverMgDeviceSelect()

```
 hipsolverStatus_t
 hipsolverMgDeviceSelect(
        hipsolverMgHandle_t handle,
       int nbDevices,
       int deviceId[] )
/*
This function registers a subset of devices (DCUs) to    hipsolverMG handle. Such subset of devices is used in subsequent API calls. The array deviceId contains a list of logical device ID. The term logical means repeated device ID are permitted. For example, suppose the user has only one DCU in the system, say device 0. If the user sets deviceId=0,0,0, then    hipsolverMG treats them as three independent DCUs, one stream each, so concurrent kernel launches still hold. The current design only supports up to 32 logical devices.
*/
```

4.4.4 hipsolverMgCreateDeviceGrid()

```
 hipsolverStatus_t
 hipsolverMgCreateDeviceGrid(
        hipsolverMgGrid_t* grid,
       int32_t numRowDevices,
       int32_t numColDevices,
       const int32_t deviceId[],
        hipsolverMgGridMapping_t mapping)
/*
This function sets up a grid of devices.
Only 1-D column block cyclic is supported, so numRowDevices must be equal to 1.
```

```
*/
```

4.4.5 hipsolverMgDestroyGrid()

```
 hipsolverStatus_t    hipsolverMgDestroyGrid( hipsolverMgGrid_t grid)
/*
This function releases resources of a grid.
*/
```

4.4.6 hipsolverMgCreateMatrixDesc()

```
 hipsolverStatus_t
 hipsolverMgCreateMatrixDesc(
        hipsolverMgMatrixDesc_t * desc,
       int64_t numRows,
       int64_t numCols,
       int64_t rowBlockSize,
       int64_t colBlockSize,
        hipDataType_t dataType,
       const    hipsolverMgGrid_t grid)
/*
This function sets up the matrix descriptor desc.
Only 1-D column block cyclic is supported, so numRows must be equal to
rowBlockSize.
*/
```

4.4.7 hipsolverMgDestroyMatrixDesc()

```
 hipsolverStatus_t
 hipsolverMgDestroyMatrixDesc(
        hipsolverMgMatrixDesc_t desc)
/*
This function releases the matrix descriptor desc.
*/
```

4.4.8 hipsolverMgPotrf()

```
hipsolverStatus_t
hipsolverMgPotrf_bufferSize(
        hipsolverMgHandle_t handle,
        hipblasFillMode_t uplo,
       int N,
       void *array_d_A[],
       int IA,
       int JA,
       hipLibMgMatrixDesc_t descrA,
        hipDataType computeType,
       int64_t *lwork);
        hipsolverStatus_t
        hipsolverMgPotrf(
              hipsolverMgHandle_t handle,
              hipblasFillMode_t uplo,
             int N,
             void *array_d_A[],
             int IA,
             int JA,
             hipLibMgMatrixDesc_t descrA,
              hipDataType computeType,
             void *array_d_work[],
             int64_t lwork,
             int *info)
/*
The function computes the Cholesky factorization of a Hermitian positive-definite matrix using the generic API interface.
A is an n × n Hermitian matrix; only lower or upper part is meaningful. The input
```

```
parameter uplo indicates which part of the matrix is used. The function would leave other parts untouched.
*/
```

4.4.9 hipsolverMgPotrs()

```
hipsolverStatus_t
hipsolverMgPotrs_bufferSize(
        hipsolverMgHandle_t handle,
        hipblasFillMode_t uplo,
      int n,
      int nrhs,
      void *array_d_A[],
      int IA,
      int JA,
      hipLibMgMatrixDesc_t descrA,
      void *array_d_B[],
      int IB,
      int JB,
      hipLibMgMatrixDesc_t descrB,
        hipDataType computeType,
      int64_t *lwork );
        hipsolverStatus_t
        hipsolverMgPotrs(
              hipsolverMgHandle_t handle,
              hipblasFillMode_t uplo,
            int n,
            int nrhs,
            void *array_d_A[],
            int IA,
            int JA,
```

```
            hipLibMgMatrixDesc_t descrA,
            void *array_d_B[],
            int IB,
            int JB,
            hipLibMgMatrixDesc_t descrB,
             hipDataType computeType,
            void *array_d_work[],
            int64_t lwork,
            int *info)
/*
This function solves a system of linear equations:
                                    A * X = B
where A is an  n × n  Hermitian matrix, only lower or upper part is meaningful using
the generic API interface. The input parameter uplo indicates which part of the matrix
is used. The function would leave other parts untouched.
*/
```

4.4.10 hipsolverMgPotri()

```
 hipsolverStatus_t
 hipsolverMgPotri_bufferSize(
       hipsolverMgHandle_t handle,
       hipblasFillMode_t uplo,
      int N,
      void *array_d_A[],
      int IA,
      int JA,
      hipLibMgMatrixDesc_t descrA,
       hipDataType computeType,
      int64_t *lwork);
       hipsolverStatus_t
```

```
        hipsolverMgPotri(
              hipsolverMgHandle_t handle,
              hipblasFillMode_t uplo,
             int N,
             void *array_d_A[],
             int IA,
             int JA,
             hipLibMgMatrixDesc_t descrA,
              hipDataType computeType,
             void *array_d_work[],
             int64_t lwork,
             int *info)
/*
This function computes the inverse of a Hermitian positive-definite matrix A using the
Cholesky factorization
                                    A = L * L^H = U^H * U
computed by    hipsolverMgPotrf().
*/
```

4.4.11 hipsolverMgGetrf()

```
 hipsolverStatus_t
 hipsolverMgGetrf_bufferSize(
        hipsolverMgHandle_t handle,
       int M,
       int N,
       void *array_d_A[],
       int IA,
       int JA,
       hipLibMgMatrixDesc_t descrA,
       int *array_d_IPIV[],
```

```
        hipDataType_t computeType,
      int64_t *lwork);
       hipsolverStatus_t
       hipsolverMgGetrf(
            hipsolverMgHandle_t handle,
          int M,
          int N,
          void *array_d_A[],
          int IA,
          int JA,
          hipLibMgMatrixDesc_t descrA,
          int *array_d_IPIV[],
           hipDataType_t computeType,
          void *array_d_work[],
          int64_t lwork,
          int *info );
/*
This function computes the LU factorization of an  M × N  matrix
                                  P ∗ A = L ∗ U
where A is an  M × N  matrix, P is a permutation matrix, L is a lower triangular matrix
with unit diagonal, and U is an upper triangular matrix.
*/
```

4.4.12 hipsolverMgGetrs()

```
 hipsolverStatus_t
 hipsolverMgGetrs_bufferSize(
       hipsolverMgHandle_t handle,
       hipblasOperation_t TRANS,
      int N,
      int NRHS,
```

```
    void *array_d_A[],
    int IA,
    int JA,
    hipLibMgMatrixDesc_t descrA,
    int *array_d_IPIV[],
    void *array_d_B[],
    int IB,
    int JB,
    hipLibMgMatrixDesc_t descrB,
     hipDataType_t computeType,
    int64_t *lwork);
     hipsolverStatus_t
     hipsolverMgGetrs(
            hipsolverMgHandle_t handle,
            hipblasOperation_t TRANS,
           int N,
           int NRHS,
           void *array_d_A[],
           int IA,
           int JA,
           hipLibMgMatrixDesc_t descrA,
           int *array_d_IPIV[],
           void *array_d_B[],
           int IB,
           int JB,
           hipLibMgMatrixDesc_t descrB,
            hipDataType_t computeType,
           void *array_d_work[],
           int64_t lwork,
```

```
            int *info );
/*
This function solves a linear system of multiple right-hand sides
                        op(A) * X = B
where A is an  N × N  matrix, and was LU-factored by getrf, that is, lower triangular
part of A is L, and upper triangular part (including diagonal elements) of A is U. B is an
N × NRHS  right-hand side matrix. The solution matrix X overwrites the right-hand-
side matrix B.
*/
```

4.4.13 hipsolverMgSyevd()

```
hipsolverStatus_t
hipsolverMgSyevd_bufferSize(
      hipsolverMgHandle_t handle,
      hipsolverEigMode_t jobz,
      hipblasFillMode_t uplo,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    hipLibMgMatrixDesc_t descrA,
    void *W,
     hipDataType_t dataTypeW,
     hipDataType_t computeType,
    int64_t *lwork
    );
     hipsolverStatus_t
     hipsolverMgSyevd(
          hipsolverMgHandle_t handle,
          hipsolverEigMode_t jobz,
```

```
            hipblasFillMode_t uplo,
            int N,
            void *array_d_A[],
            int IA,
            int JA,
            hipLibMgMatrixDesc_t descrA,
            void *W,
            hipDataType_t dataTypeW,
            hipDataType_t computeType,
            void *array_d_work[],
            int64_t lwork,
            int *info );
/*
This function computes eigenvalues and eigenvectors of a symmetric (Hermitian)
 matrix A. The standard symmetric eigenvalue problem is:
                                A * V = V * Λ
where Λ is a real N × N diagonal matrix. V is an N × N unitary matrix. The diagonal
elements of Λ are the eigenvalues of A in ascending order.
*/
```

5 调试与分析

5.1 环境变量

hipSOLVER 的环境变量主要用于 Log 输出，且沿用 rocSOLVER 的环境变量名称和用法：

- ROCSOLVER_LAYER
  - 0：layer_mode_none
    不进行 log 输出。
  - 1：layer_mode_log_trace
    每次调用一个 rocSOLVER 函数时输出一行，包含函数名和传递参数

值。

- 2：layer_mode_log_bench
  每次调用一个 rocSOLVER 函数时输出一行，内容是使用 rocsolver -bench 复现该函数调用的命令。
- 4：layer_mode_log_profile
  输出所调用的每个 rocSOLVER 函数的 YAML 描述，以及参数和调用次数。

- ROCSOLVER_LEVELS
  rocSOLVER log 输出时，函数调用深度的最大层级。
- ROCSOLVER_LOG_TRACE_PATH
  layer_mode_log_trace 的输出路径，默认标准错误输出
- ROCSOLVER_LOG_BENCH_PATH
  layer_mode_bench_trace 的输出路径，默认标准错误输出
- ROCSOLVER_LOG_PROFILE_PATH
  layer_mode_profile_trace 的输出路径，默认标准错误输出

开发者社区

保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区